

Heterogeneous agent models by deep learning (Krusell-Smith, discrete labor choice and HANK)

Serguei Maliar

June 6

PKU-Zurich PhD Summer School on
Machine Learning for Macroeconomics and Finance
July 6–10, 2026 · Beijing, China

Why are heterogenous models interesting?

- Representative agent models dominate macroeconomics and provide a tractable way to model aggregate time series.
- But the economists are also interested in distributions.
- For example, if policymakers increase the nominal interest rate, how would this affects:
 - rich versus poor agents?
 - renters versus home owners?
 - old versus young agents?
 - high income versus low income groups? etc.
- Representative model can't answer these questions.
- We need a heterogeneous agent models that produces distributions.

What heterogenous models do we consider?

- Idiosyncratic and aggregate shocks, incomplete markets and borrowing constraints, see Krusell and Smith (1988).
 - The economy has ℓ heterogeneous agents that differ in capital and productivity that have a joint distribution $\{k_t^i, y_t^i\}_{i=1}^{\ell}$ for $i = 1, \dots, \ell$.
 - The economy has also aggregate productivity z_t .
 - Thus, the economy has $2\ell + 1$ state variables $\left(\{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right)$, for example, if $\ell = 1,000$ the state space has 2001 state variables.
- HANK is heterogeneous-agent new Keynesian model that includes monetary policy, for example, Taylor rule for the nominal interest rate.

Agenda 1. Curse of dimensionality

- In representative agent growth model, we construct decision functions of two arguments - capital and productivity (k_t, z_t)

$$c_t = C(k_t, z_t), \text{ and } k_{t+1} = K(k_t, z_t).$$

Even this is a non entirely trivial task: Recall dynamic programming methods iterating on Bellman equation or Euler equations.

- In a HA model, we construct decision functions of $2\ell + 1$ arguments for $i = 1, \dots, \ell$ agents:

$$c_t^i = C\left(\{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right) \text{ and } k_{t+1}^i = K\left(\{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right).$$

- Such decision and value functions can depend literally on thousands of state variables, recall that if $\ell = 1,000$, we have 2001 state variables.
- Severe curse of dimensionality: computational expense increases exponentially!

Agenda 2. Discrete-set choices

- Macroeconomic models are generally built on continuous-set choices.
- For example, the agent can distribute wealth **in any proportion** between consumption and savings or she can distribute time endowment in any proportion between work and leisure.
- But certain economic choices are **discrete**: the agent can either buy a house or not, be either employed or not, either retire or not, etc.
- In those models, we can't even use Euler equations, the decision functions have discontinuities and jumps, etc.
- Modeling discrete-set choices is challenging.

Deep learning for solving heterogeneous-agent models

- We apply deep learning methods from data science to solve high-dimensional models with both continuous-set and discrete-set choices.
- To solve for **continuous-set choices**:
 - we parameterize decision functions with a deep neural network;
 - and we find the coefficients of the neural network (biases and weights) to satisfy the model's equations.
- To solve for **discrete-set choices**:
 - We define a state-contingent probability function that for each discrete choice, gives the probability that a given choice is optimal;
 - we parameterize the probability function with a deep neural network;
 - and we find the network parameters to satisfy the optimality conditions for the discrete choices.

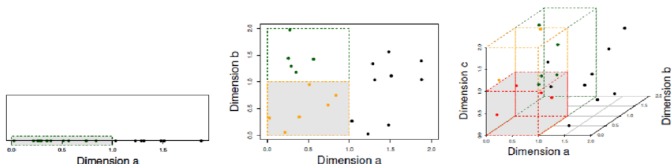
The structure of the presentation

- Part 1: We analyze the model with **just continuous-set choices** - we focus on the curse of dimensionality.
 - Krusell and Smith (1998) model
 - HANK model
- Part 2: We augment the model to include **discrete-set choices**, namely, employment choices (employed, unemployed, part-time job).

Multiple curses of dimensionality

There are multiple curses of dimensionality in economic models:

1. The number of grid points increases exponentially with the number of state variables. For example, if we have $k_1^1, \dots, k_{10}^1$ for agent 1 and $k_1^2, \dots, k_{10}^2$ for agent 2, we get $10^2 = 100$ grid points for 2 agents, $10^3 = 1000$ for 3 agents, and 10^ℓ for ℓ agents:



2. Number of polynomial coefficients grows exponentially with the number of grids points.
3. The number of integration nodes increases exponentially with number of shocks. For example, if we have $\epsilon_1^1, \dots, \epsilon_{10}^1$ for shock 1 and $\epsilon_1^2, \dots, \epsilon_{10}^2$ for shock 2, we get $10^2 = 100$ integration nodes, and 10^ℓ grid points for with ℓ agents.
4. The cost of iterating on value function and Euler equations also increases rapidly with dimensionality.

Solution method of Krusell and Smith (1998)

- Krusell and Smith (1998) analyze a heterogeneous-agent neoclassical model in which the agents save through capital.
- Such a model has a remarkable feature of "approximate aggregation" – the mean of future capital distribution can be accurately predicted just with the mean of the current capital distribution.
- Approximate aggregation allows to deal with the curse of dimensionality in their model by reducing the number of state variables from thousands to just a few (four or five).
- Krusell and Smith (1998) method was successfully used to solve numerous heterogeneous agent models.

Krusell-Smith (1998) model with savings through capital

- Heterogeneous agents $i = 1, \dots, \ell$ solve

$$\begin{aligned} & \max_{\{c_t^i, k_{t+1}^i\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t^i) \right] \\ & \text{s.t. } c_t^i + k_{t+1}^i = R_t k_t^i + W_t y_t^i, \end{aligned}$$

where $c_t^i, k_t^i \geq 0$ are consumption and capital; idiosyncratic labor productivity follows $\ln y_{t+1}^i = \rho_y \ln y_t^i + \sigma_y \epsilon_t^i$ with $\epsilon_t^i \sim \mathcal{N}(0, 1)$; and $\beta \in (0, 1)$; $\rho_y \in (-1, 1)$ and $\sigma_y \geq 0$; and (k_0^i, y_0^i) is given.

- The Cobb-Douglas production implies the interest rate R_t and wage W_t

$$R_t = 1 - d + z_t \alpha k_t^{\alpha-1} h_t^{1-\alpha} \text{ and } W_t = z_t (1 - \alpha) k_t^{\alpha} h_t^{-\alpha},$$

where $d \in (0, 1]$ is the depreciation rate, $k_t = \sum_{i=1}^{\ell} k_t^i$ is aggregate capital, $h_t = \sum_{i=1}^{\ell} y_t^i$ is aggregate efficiency labor, and aggregate shock follows $\ln z_{t+1} = \rho_z \ln z_t + \sigma_z \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, 1)$, $\rho_z \in (-1, 1)$ and $\sigma_z \geq 0$.

Krusell-Smith (1998) method with savings through capital

- **The key idea:** Replace a large distribution $\{k_t^i, y_t^i\}_{i=1}^\ell$ with a relatively small set of moments m_t .
- **Approximate aggregation:** it turned out that 1st moment contains essentially all information about the distributions, i.e., $m_t = k_t$.
- Construct separately the individual and aggregate decision functions:

Individual: Find $k_{t+1}^i = K(k_t^i, y_t^i, z_t, m_t)$ given $m_{t+1} = G(m_t, z_t)$,
Aggregate: Find $m_{t+1} = G(m_t, z_t)$ given $k_{t+1}^i = K(k_t^i, y_t^i, z_t, m_t)$,

where m_t is a selected set of aggregate statistics such as moments.

- K&S (1998) method alternates between
 - solving the agents problem with 4 state variables (k_t^i, y_t^i, z_t, m_t) and
 - constructing the aggregate law of motion with 2 state variables (z_t, m_t) .
- **Shall we declare a victory?** Not really.

Krusell-Smith (1998) model with savings through bonds

- Heterogeneous agents $i = 1, \dots, \ell$ solve

$$\begin{aligned} \max_{\{c_t^i, b_{t+1}^i\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t^i) \right] \\ \text{s.t. } c_t^i + q_t b_{t+1}^i = b_t^i + y_t^i z_t, \end{aligned}$$

where $c_t^i, b_{t+1}^i \geq \underline{b}$ are consumption and bonds; idiosyncratic labor productivity follows $\ln y_{t+1}^i = \rho_y \ln y_t^i + \sigma_y \epsilon_t^i$ with $\epsilon_t^i \sim \mathcal{N}(0, 1)$; and $\beta \in (0, 1)$; $\rho_y \in (-1, 1)$ and $\sigma_y \geq 0$; and (b_0^i, y_0^i) is given and the aggregate shock follows $\ln z_{t+1} = \rho_z \ln z_t + \sigma_z \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, 1)$, $\rho_z \in (-1, 1)$ and $\sigma_z \geq 0$.

- The market clearing price q_t implies that the supply and demand of bonds are equal meaning that the aggregate bond holding is zero:

$$\int_{\underline{b}}^{\infty} b_{t+1}^i(b_t^i, y_t^i, z_t) dF_t(b_t^i, y_t^i) = 0$$

where $F_t(b_t^i, y_t^i)$ is the cross sectional cumulative joint distribution function of beginning-of-period bond holdings and income.

Krusell-Smith (1998) method with savings through bonds

- The state space of the economy with $i = 1, \dots, \ell$ agents has $2\ell + 1$ state variables $\left(\{b_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right)$, for example, if $\ell = 1,000$ the state space has 2001 state variables.
- In line with Krusell and Smith (1998), we can try to approximate the solution by constructing separately the individual and aggregate decision functions

Individual: Find $b_{t+1}^i = K(b_t^i, y_t^i, z_t, m_t)$ given $m_{t+1} = G(m_t, z_t)$

Aggregate: Find $m_{t+1} = G(m_t, z_t)$ given $b_{t+1}^i = K(b_t^i, y_t^i, z_t, m_t)$

where m_t is a selected set of aggregate statistics such as moments.

- But the first moment of bonds is zero $\overline{b_{t+1}^i} = 0$.
- We tried to include higher moments and statistics in $m_t \equiv \{\text{variance, skewness, kurtosis, ...}\}$
- But these moments contain limited information about distribution and the approximation $m_{t+1} = G(m_t, z_t)$ has low explanatory power.
- **Approximate aggregation does not work for the model with savings through bonds.**

Deep learning analysis of Maliar, Maliar and Winant (2021)

Maliar, Maliar and Winant (2021) propose a deep learning method that constructs decision function in terms of the actual state space

$$c_t^i = C \left(\{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t \right) \text{ and } k_{t+1}^i = K \left(\{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t \right)$$

To deal with so huge dimensionality, our DL method use:

1. stochastic simulation that allows us to restrict attention to the ergodic set in which the solution "lives";
2. multilayer neural networks that perform model reduction and help deal with multicollinearity;
3. a (batch) stochastic gradient descent method that reduces the number of function evaluations by operating on random grids;
4. a Fischer-Burmeister function that effectively approximates the kink;
5. most importantly, "all-in-one expectation operator" that allows us to approximate high-dimensional integrals with just 2 random draws (or batches) on each iteration.
6. TensorFlow – a Google data science platform that is used to facilitate the remarkable data-science applications such as image and speech recognition, self driving cars, etc.

Stochastic simulation - ergodic set domain

- Under normally distributed shocks, stochastic simulation typically have a shape of a hypersphere (hyperoval)

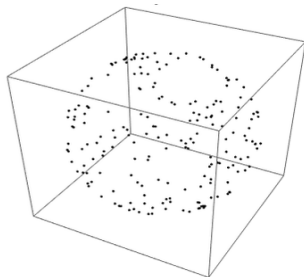


Figure 1. Hypercube versus hypersphere.

- The ratio of a volume of a hypersphere to that of an enclosing hypercube is an infinitesimally small number in high-dimensional applications; for example, for a 30-dimensional case, it is 10^{-14} ; see Judd, Maliar and Maliar (2011) for a discussion.

Neural networks

We use neural networks for parameterizing decision and value functions instead of more conventional approximation families like polynomial functions:

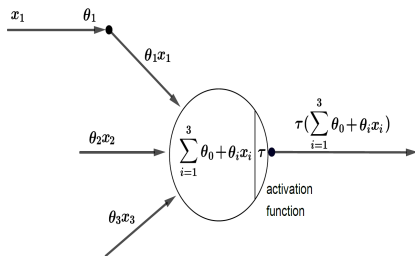


Figure 2a. Artificial neuron.

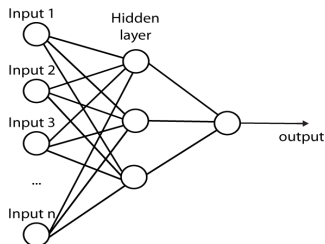


Figure 2b. Neural network.

In Figure 1a, the circle represents an artificial neuron that receives 3 signals (inputs) x_1 , x_2 and x_3 . In Figure 1b, we combine multiple neurons into a neural network.

Activation functions

The activation function that we use in our benchmark experiments is a sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-\theta_0+\theta_1x_1+\theta_2x_2+\dots+\theta_nx_n}}$.

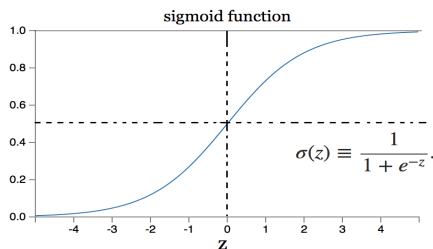


Figure 3. Sigmoid function.

The sigmoid function has two properties: First, its derivative can be inferred from the function itself $\sigma'(x) = \sigma(x)(1 - \sigma(x))$. Second, it maps a real line into a unit interval $\sigma : \mathbb{R}^n \rightarrow [0, 1]$ which makes it bounded between 0 and 1.

Parameterization of decision functions

- We solve for the fraction of wealth that goes to consumption $\frac{c_t^i}{w_t^i}$ which we parameterized by a sigmoid function

$$\sigma \left(\zeta_0 + \varphi \left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta \right) \right),$$

where $\varphi(\cdot)$ is a multilayer neural network parameterized by a vector of coefficients θ (weights and biases), $\sigma(z) = \frac{1}{1+e^{-z}}$ is a sigmoid function and ζ_0 is a constant term.

- In addition, we parameterize the Lagrange multiplier μ_t^i associated with the borrowing constraint using an exponential activation function

$$\exp \left(\zeta_0 + \varphi \left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta \right) \right).$$

The exponential activation function ensures that the Lagrange multiplier is always non-negative.

- Since the agents are identical in fundamentals, the above three $2\ell + 1$ -dimensional decision functions are sufficient to characterize the choices of all ℓ heterogeneous agents.

Model reduction

- Our DL method condenses the information from a large number of inputs into a smaller number of neurons in the hidden layers, making it progressively more abstract and compact.
- This procedure is similar to a photo compression or principal component transformation when a large dataset is condensed into a smaller set of principal components without losing essential information; see Judd, Maliar and Maliar (2011) for a discussion of model reduction using principal-component analysis.
- Krusell and Smith (1998) proposed one specific model reduction method, namely, they approximate the distribution with just one moment – the mean.
- If Krusell and Smith's (1998) analysis is the most efficient representation of the state space, the neural network will also find it.
- However, the neural network will consider many other possible ways of extracting the information from the distributions and condensing it in a relatively small set of hidden layers trying to find the best one.

Dealing with Kuhn Tucker conditions

- The objective is to minimize the squared residuals in the model's conditions:

$$\Xi(\theta) \equiv E_{(K_t, Y_t, z_t)} \left\{ \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - \mu_t^i \right) \right]^2 + \varpi_\mu \left[\frac{\beta E_{(\Sigma_{t+1}, \epsilon_{t+1})} \left[(c_{t+1}^i)^{-\gamma} R_{t+1} \mid \Sigma_{t+1}, \epsilon_{t+1} \right]}{(c_t^i)^{-\gamma}} - \mu_t^i \right]^2 \right\},$$

where $K \equiv (k^1, \dots, k^\ell)$ and $Y \equiv (y^1, \dots, y^\ell)$ are state variables; z_t is aggregate productivity; $\Sigma_{t+1} \equiv (\epsilon_{t+1}^1, \dots, \epsilon_{t+1}^\ell)$ the individual productivity shocks; ϵ_{t+1} is the aggregate productivity shock; and

$$\Psi^{FB}(a, b) = a + b - \sqrt{a^2 + b^2},$$

is a $\Psi^{FB}(a, b) = 0$ is a Fisher-Burmeister objective function is equivalent to Kuhn Tucker conditions.

Nested expectation operators

- The objective $\Xi(\theta)$ is not tractable because it contains a square of expectation $\left[E_{(\Sigma_{t+1}, \epsilon_{t+1})} [\cdot]\right]^2$ nested inside another expectation $E_{(K_t, Y_t, z_t)} [\cdot]$.
- Constructing two nested expectation operators is costly because $E_{(\Sigma_{t+1}, \epsilon_{t+1})} [\cdot]$ has high dimensionality; e.g., if $\ell = 1,000$, it is 1,001-dimensional integral.
- This task would be simplified enormously if combine the two expectation operators but it is not possible

$$E_{(K_t, Y_t, z_t)} \left[\left[E_{(\Sigma_{t+1}, \epsilon_{t+1})} [\cdot] \right]^2 \right] \neq E_{(K_t, Y_t, z_t)} E_{(\Sigma_{t+1}, \epsilon_{t+1})} \left[[\cdot]^2 \right].$$

All in one expectation operator

- Maliar et al. (2021) propose a critical technique, called *all-in-one* (AiO) expectation operator, that can effectively combine the two expectation operators into one.

$$\text{AiO: } \left[E_{(\Sigma_{t+1}, \epsilon_{t+1})} [\cdot] \right]^2 \approx \left[E_{(\Sigma'_{t+1}, \epsilon'_{t+1})} [\cdot] \right] \times \left[E_{(\Sigma''_{t+1}, \epsilon''_{t+1})} [\cdot] \right].$$

i.e., they replace the squared expectation function $\left[E_{(\Sigma_{t+1}, \epsilon_{t+1})} [\cdot] \right]^2$ under one random draw $(\Sigma_{t+1}, \epsilon_{t+1})$ with a product of two expectation functions $\left[E_{(\Sigma'_{t+1}, \epsilon'_{t+1})} [\cdot] \right] \times \left[E_{(\Sigma''_{t+1}, \epsilon''_{t+1})} [\cdot] \right]$ under two uncorrelated random draws $(\Sigma'_{t+1}, \epsilon'_{t+1})$ and $(\Sigma''_{t+1}, \epsilon''_{t+1})$.

- Since the two random draws are uncorrelated, the expectation operator can be taken outside of the expectation function.

The objective function under AiO expectation operator

We can represent the objective function as follows:

$$\Xi(\theta) \equiv E_{(K_t, Y_t, z_t, \Sigma'_{t+1}, \epsilon'_{t+1}, \Sigma''_{t+1}, \epsilon''_{t+1})} \left\{ \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - \mu_t^i \right) \right]^2 + \varpi_\mu \times \right. \\ \left. \left[\frac{\beta \left[(c_{t+1}^i)^{-\gamma} R_{t+1} \middle| \Sigma'_{t+1}, \epsilon'_{t+1} \right]}{(c_t^i)^{-\gamma}} - \mu_t^i \right] \left[\frac{\beta \left[(c_{t+1}^i)^{-\gamma} R_{t+1} \middle| \Sigma''_{t+1}, \epsilon''_{t+1} \right]}{(c_t^i)^{-\gamma}} - \mu_t^i \right] \right\}$$

Thus, we are able to represent the studied model as an expectation function across a vector of random variables

$(K_t, Y_t, z_t, \Sigma'_{t+1}, \epsilon'_{t+1}, \Sigma''_{t+1}, \epsilon''_{t+1})$; see Maliar et al. (2021) for a discussion and further applications of the AiO expectation operator.

Training: gradient descent, batches and parallel computing

- Given that AiO is an expectation function, we can bring the gradient operator inside by writing $\nabla_{\theta}\Xi(\theta) = \nabla_{\theta}E[\xi(\omega; \theta)] = E[\nabla_{\theta}\xi(\omega; \theta)]$, where ∇_{θ} is a gradient operator.
- The latter expectation function can be approximated by a simple average across Monte Carlo random draws $E[\nabla_{\theta}\xi(\omega; \theta)] \approx \frac{1}{N} \sum_{n=1}^N \nabla_{\theta}\xi(\omega_n; \theta)$, where ω_n denotes a specific realization of the vector of random variables.
- The gradient descent method can be implemented as

$$\theta \leftarrow \theta - \lambda \nabla_{\theta}\Xi(\theta) \quad \text{with} \quad \nabla_{\theta}\Xi(\theta) \approx \frac{1}{N} \sum_{n=1}^N \nabla_{\theta}\xi(\omega_n; \theta),$$

where θ and λ are the parameter vector and learning rate, respectively.

- Thus, we implement a cheap computation of the gradient of the integrand instead of computing far more expensive gradient of the expectation function. TensorFlow and PyTorch can compute such a gradient using a symbolic differentiation, which facilitates the implementation of parallel computation.

Dealing with multicollinearity

- In the arguments of approximating functions, the state variables of agent i appear twice $\varphi\left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta\right)$ because they enter both as variables of agent i and as an element of the distribution.
- This repetition implies perfect collinearity in explanatory variables, so that the inverse problem is not well defined.
- Such a multicollinearity would break down a conventional least-squares method which solves the inverse problem (since an inverse of a matrix with linearly dependent rows or columns does not exist).
- However, neural networks are trained by using the gradient-descent method that avoids solving an inverse problem. As a result, neural networks can learn to ignore redundant colinear variables; see Maliar et al. (2021) for numerical illustrations and a discussion.

Algorithm 1: Deep learning for model with capital

Algorithm 1: Deep learning for the model with capital

Step 0: (Initialization).

Construct initial state of the economy $\left(\{k_0^i, y_0^i\}_{i=1}^{\ell}, z_0\right)$ and parameterize three decision functions by a neural network with three outputs

$$\left\{\frac{c_t^i}{w_t^i}\right\} = \sigma\left(\zeta_0 + \varphi\left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t; \theta\right)\right),$$
$$\mu_t^i = \exp\left(\zeta_0 + \varphi\left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^{\ell}, z_t; \theta\right)\right),$$

where $w_t^i \equiv R_t k_t^i + W_t y_t^i n_t^i$ is wealth; μ_t^i is Lagrange multiplier associated with the borrowing constraint; $\varphi(\cdot)$ is a neural network; $\sigma(z) = \frac{1}{1+e^{-z}}$ is a sigmoid (logistic) function; ζ_0 is a constant; θ is a vector of coefficients.

Algorithm 1: Deep learning for the model with capital

Algorithm 1: Deep learning for the model with capital

Step 1: (Evaluation of decision functions).

Given state $\left(k_t^i, y_t^i, \{k_t^i, y_t^i\}_{i=1}^\ell, z_t\right) \equiv s_t^i$, compute $\mu_t^i, \frac{c_t^i}{w_t^i}$ from the neural networks, find the prices R_t and W_t ; and find k_{t+1}^i from the budget constraint for all agents $i = 1, \dots, \ell$.

Step 2: (Construction of Euler residuals).

Draw two random sets of individual productivity shocks $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^\ell)$, $\Sigma_2 = (\epsilon_2^1, \dots, \epsilon_2^\ell)$ and two aggregate shocks ϵ_1, ϵ_2 , and construct Euler residuals

$$\Xi(\theta) = \left\{ \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - \mu_t^i \right) \right]^2 + \varpi_\mu \left[\frac{\beta \left[(c_{t+1}^i)^{-\gamma} R_{t+1} \middle| \Sigma'_{t+1}, \epsilon'_{t+1} \right]}{(c_t^i)^{-\gamma}} - \mu_t^i \right] \left[\frac{\beta \left[(c_{t+1}^i)^{-\gamma} R_{t+1} \middle| \Sigma''_{t+1}, \epsilon''_{t+1} \right]}{(c_t^i)^{-\gamma}} - \mu_t^i \right] \right\},$$

where ϖ_n, ϖ_μ are given weights and $\Psi^{FB}(a, b) = a + b - \sqrt{a^2 + b^2}$ is a Fischer-Burmeister function.

Algorithm: Deep learning for the model with capital

Algorithm 1: Deep learning for the model with capital

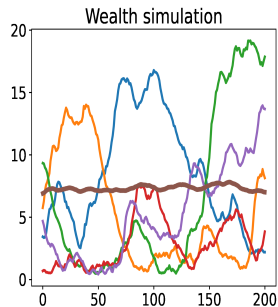
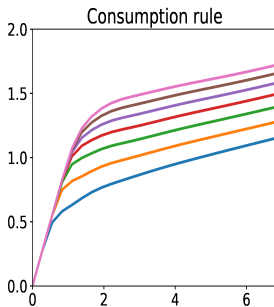
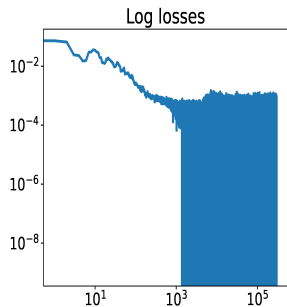
Step 3: (Training).

Train the neural network coefficients θ to minimize the residual function $\Xi(\theta)$ by using a stochastic gradient descent method $\theta \leftarrow \theta - \lambda \nabla_{\theta} \Xi(\theta)$ with $\nabla_{\theta} \Xi(\theta) \approx \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \xi(\omega_n; \theta)$, where $n = 1, \dots, N$ denotes batches.

Step 4: (Simulation).

Move to $t + 1$ by using endogenous and exogenous variables of Step 3 under $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^{\ell})$ and ϵ_1 as a next-period state $\left(\{k_{t+1}^i, y_t^i\}_{i=1}^{\ell}, z_{t+1} \right)$.

Solution to the model with savings through capital



Deep learning method for the model with bonds

- We apply this DL method and solve for decision function in terms of the actual state space $\left(\{b_t^i, y_t^i\}_{i=1}^\ell, z_t\right)$.
- We parameterize the individual variables consumption share and the Lagrange multiplier by neural networks:

$$\frac{c_t^i}{w_t^i} \equiv \varphi\left(\{b_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta_1\right) \text{ and } \eta_t^i = \varphi\left(\{b_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta_2\right).$$

- We also parameterize the bond price function q with a different neural network

$$q_t = Q\left(\{b_t^i, y_t^i\}_{i=1}^\ell, z_t; \theta_3\right).$$

- We just simulate the economy forward and construct the above functions in terms of actual state space to find the coefficients θ_1 , θ_2 and θ_3 of the neural network.
- Conceptually, our solution procedure is simpler than Krusell and Smith (1998) method because we do not separate between the individual and aggregate decision functions.

Algorithm 2: Deep learning for the model with bonds

Algorithm 2: Deep learning for the model with bonds

Step 0: (Initialization).

Construct initial state of the economy $\left(\{k_0^i, y_0^i\}_{i=1}^{\ell}, z_0\right)$ and parameterize three decision functions by a neural network with three outputs

$$\begin{aligned}\frac{c_t^i}{w_t^i} &= \sigma \left(\zeta_0 + \varphi \left(k_t^i, v_t^i, \{k_t^i, v_t^i\}_{i=1}^{\ell}, z_t; \theta_1 \right) \right), \\ \mu_t^i &= \exp \left(\zeta_0 + \varphi \left(k_t^i, v_t^i, \{k_t^i, v_t^i\}_{i=1}^{\ell}, z_t; \theta_2 \right) \right), \\ q_t &= \sigma \left(\zeta_0 + \varphi \left(\{k_t^i, v_t^i\}_{i=1}^{\ell}, z_t; \theta_3 \right) \right)\end{aligned}$$

where μ_t^i is Lagrange multiplier associated with the borrowing constraint; $\varphi(\cdot)$ is a neural network; $\sigma(z) = \frac{1}{1+e^{-z}}$ is a sigmoid (logistic) function; ζ_0 is a constant; θ is a vector of coefficients.

Algorithm 2: Deep learning for the model with bonds

Algorithm 2: Deep learning for the model with bonds

Step 1: (Evaluation of decision functions).

Given state $\left(b_t^i, y_t^i, \{b_t^i, y_t^i\}_{i=1}^\ell, z_t\right) \equiv s_t^i$, compute $q_t, \mu_t^i, \frac{c_t^i}{w_t^i}$ from the neural networks, find b_{t+1}^i from the budget

constraint for all agents $i = 1, \dots, \ell$.

Step 2: (Construction of Euler residuals).

Draw two random sets of individual productivity shocks $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^\ell)$, $\Sigma_2 = (\epsilon_2^1, \dots, \epsilon_2^\ell)$ and two aggregate shocks ϵ_1, ϵ_2 , and construct Euler residuals

$$E_{(B,Y,z,\Sigma_1,\Sigma_2,\epsilon_1,\epsilon_2)}\Xi(\theta) = \left\{ \left[\Psi \left(\frac{w^i - q_t \bar{b}}{c^i} - 1, \eta^i - 1 \right) \right]^2 + \frac{v_2}{\ell} \sum_{i=1}^\ell (b^{i'})^2 \right. \\ \left. + v_1 \left[\beta \left[\frac{\beta[u'(c^{i'})]}{qu'(c^i)} \right] \Big|_{\Sigma=\Sigma_1, \epsilon=\epsilon_1} - \eta^i \right] \left[\beta \left[\frac{\beta[u'(c^{i'})]}{qu'(c^i)} \right] \Big|_{\Sigma=\Sigma_1, \epsilon=\epsilon_1} - \eta^i \right] \right\},$$

where v_1, v_2 are given weights and $\Psi^{FB}(a, b) = a + b - \sqrt{a^2 + b^2}$ is a Fischer-Burmeister function.

Algorithm 2: Deep learning for the model with bonds

Algorithm 2: Deep learning for the model with bonds
Step 3: (Training).
Train the neural network coefficients θ to minimize the residual function $\Xi(\theta)$ by using a stochastic gradient descent method $\theta \leftarrow \theta - \lambda \nabla_{\theta} \Xi(\theta)$ with $\nabla_{\theta} \Xi(\theta) \approx \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \xi(\omega_n; \theta)$, where $n = 1, \dots, N$ denotes batches.
Step 4: (Simulation).
Move to $t + 1$ by using endogenous and exogenous variables of Step 3 under $\Sigma_1 = (\epsilon_1^1, \dots, \epsilon_1^{\ell})$ and ϵ_1 as a next-period state $\left(\{b_{t+1}^i, y_t^i\}_{i=1}^{\ell}, z_{t+1} \right)$.

Two neural networks with different topologies

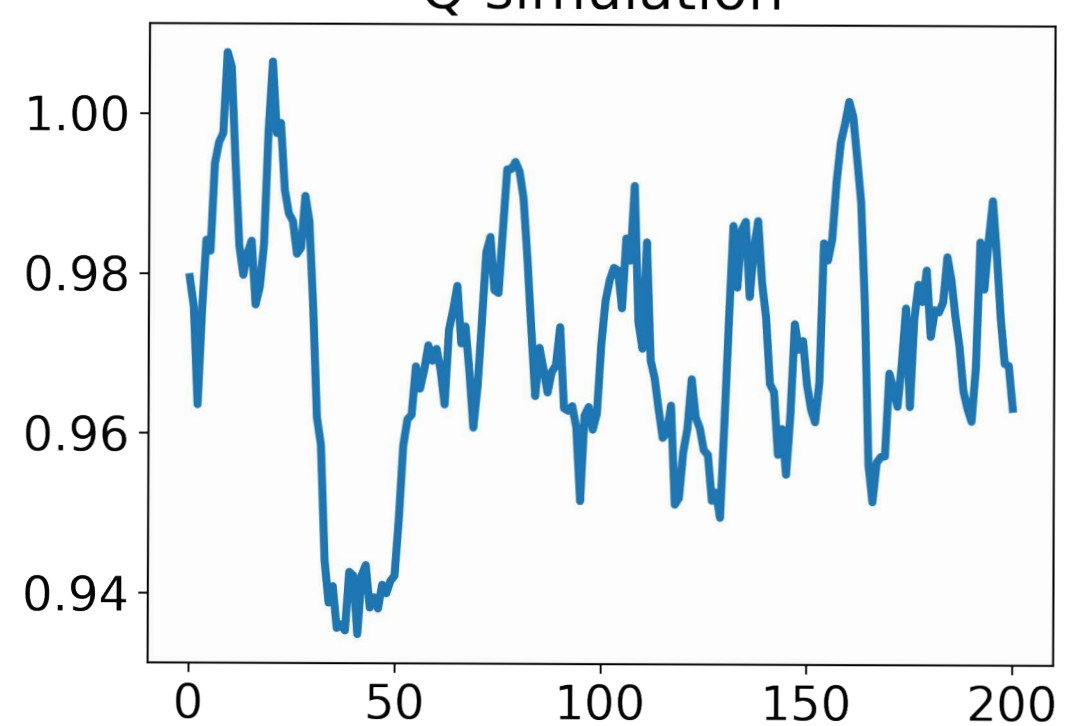
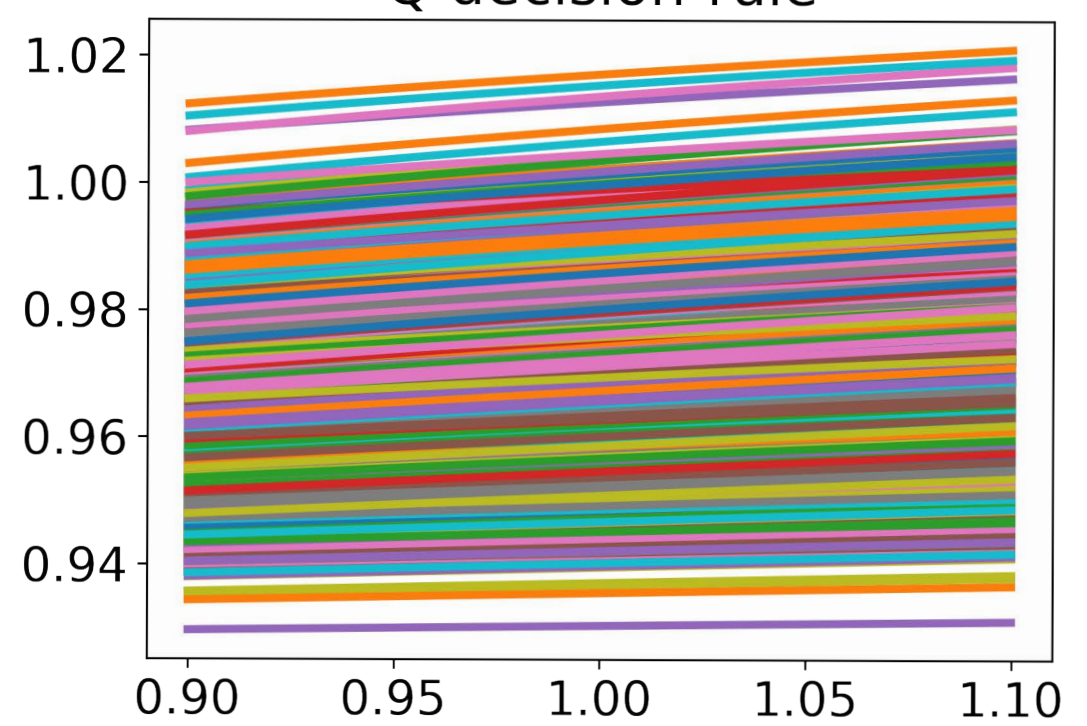
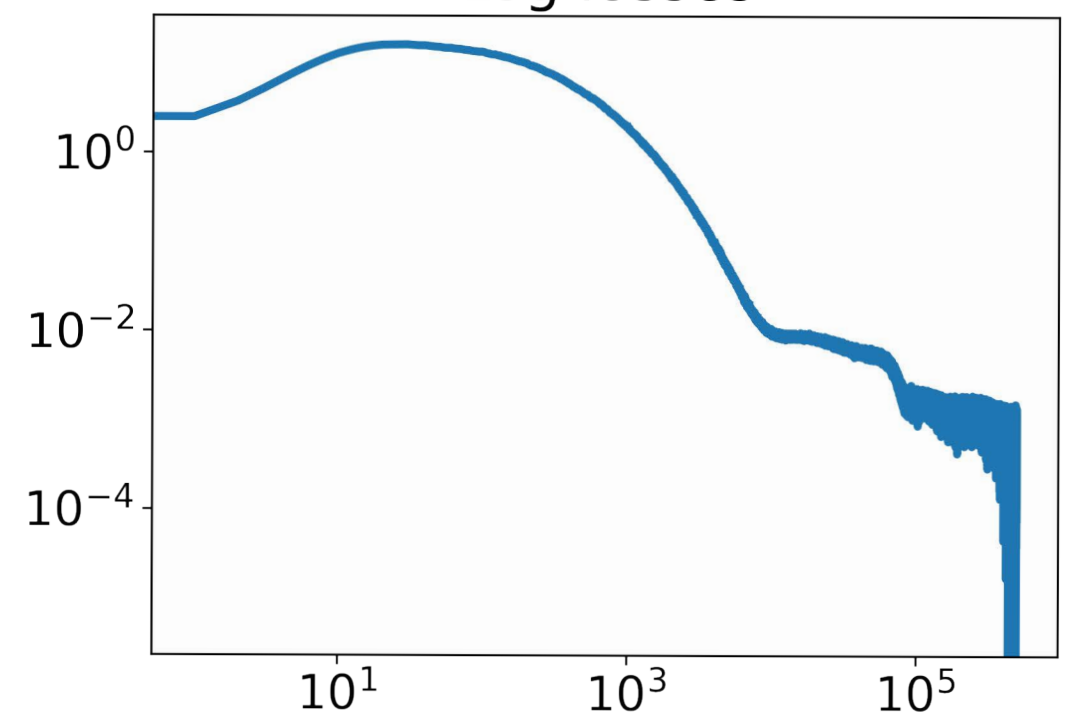
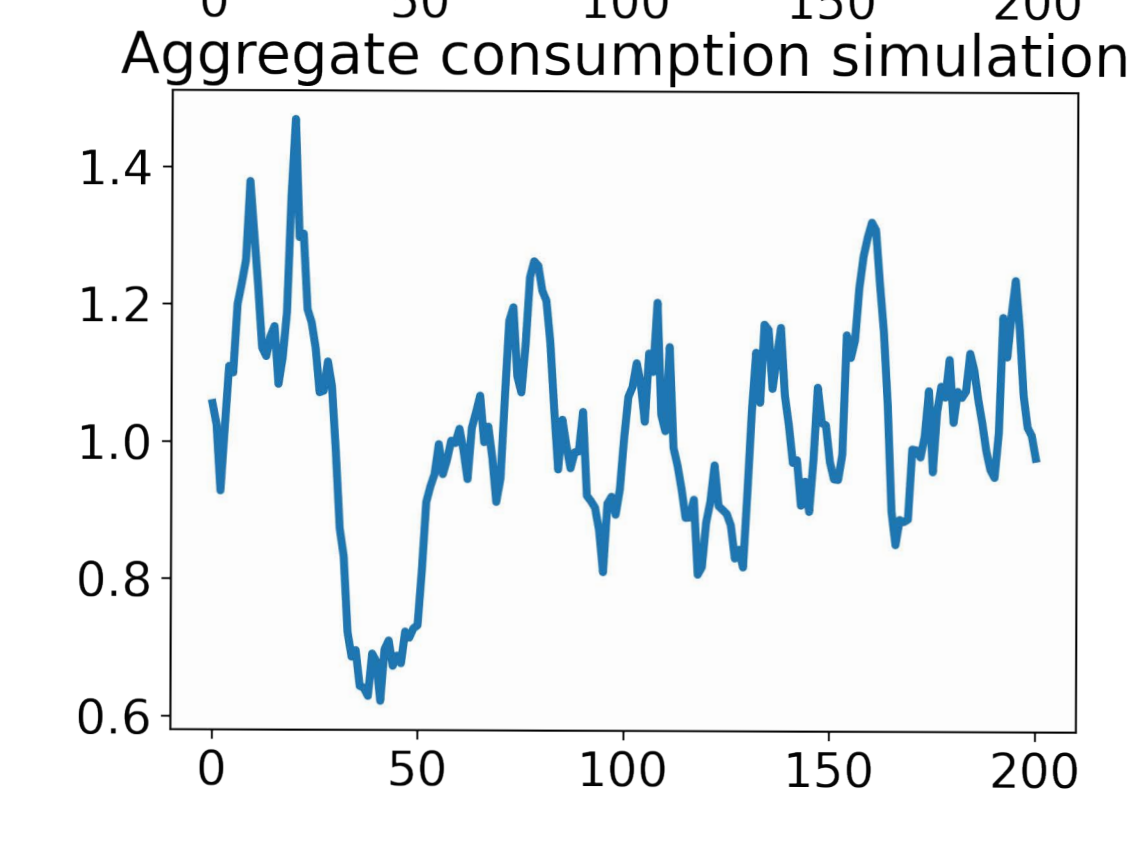
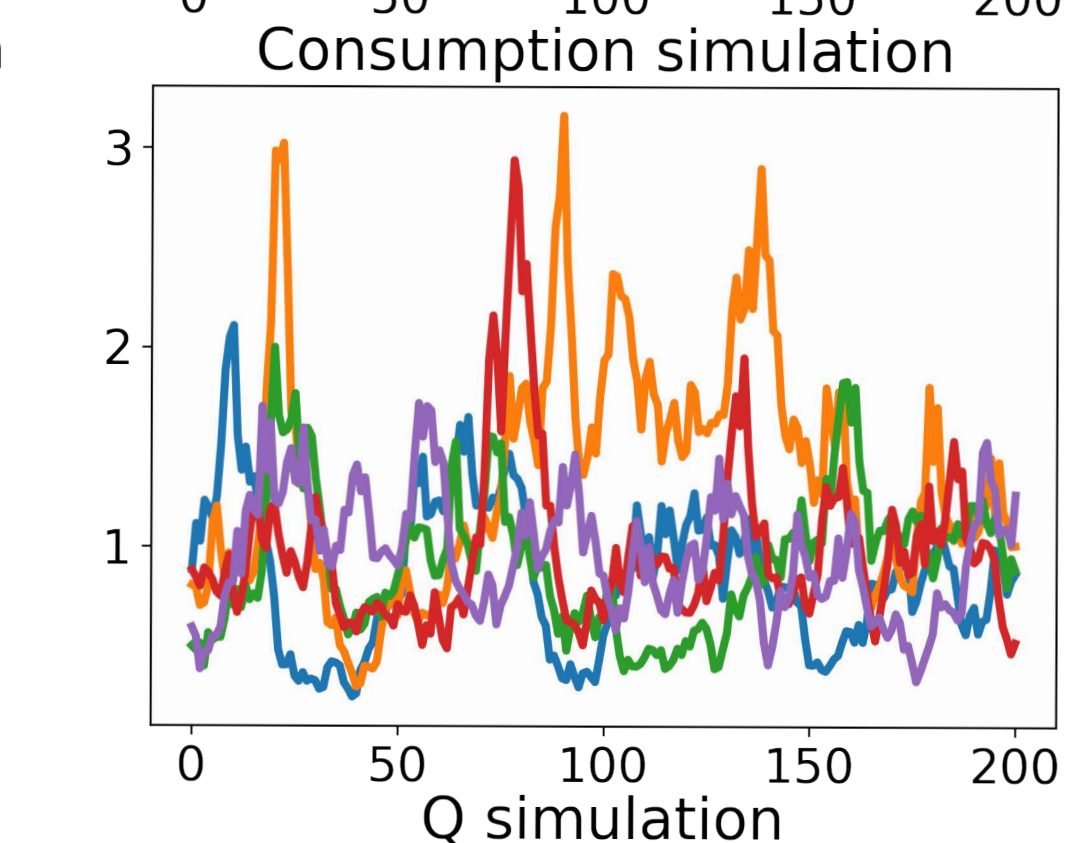
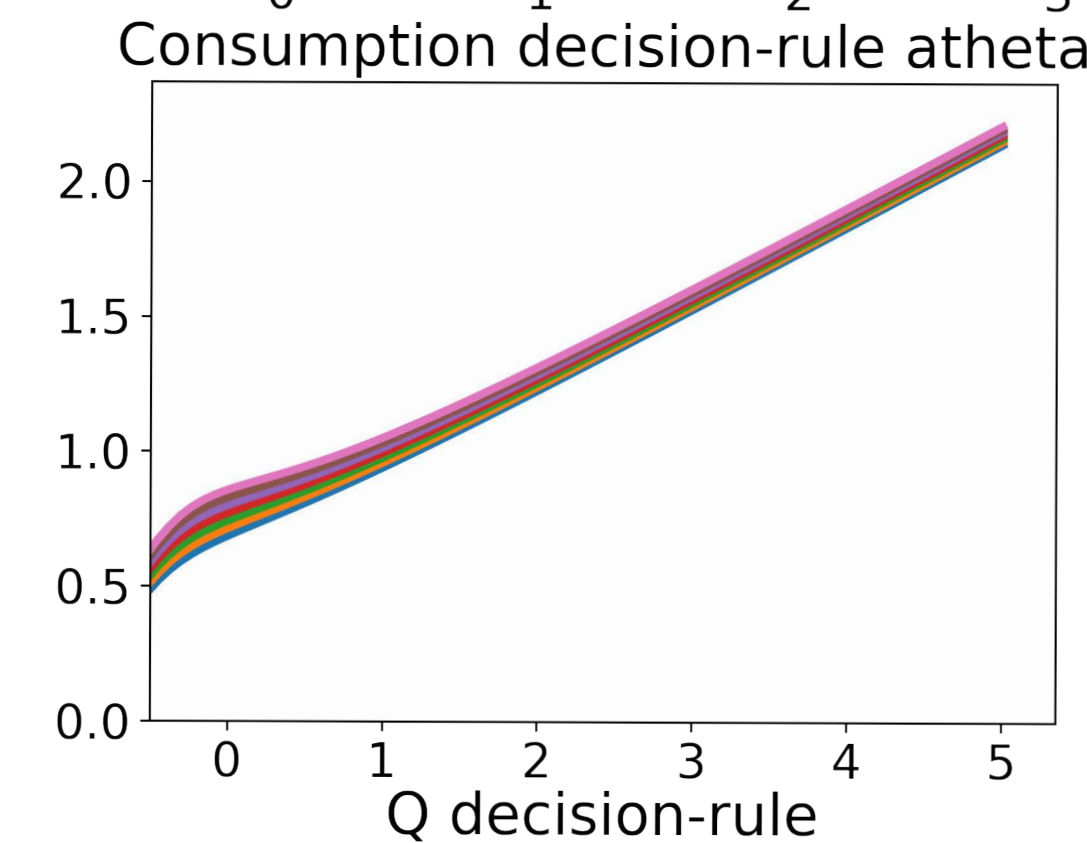
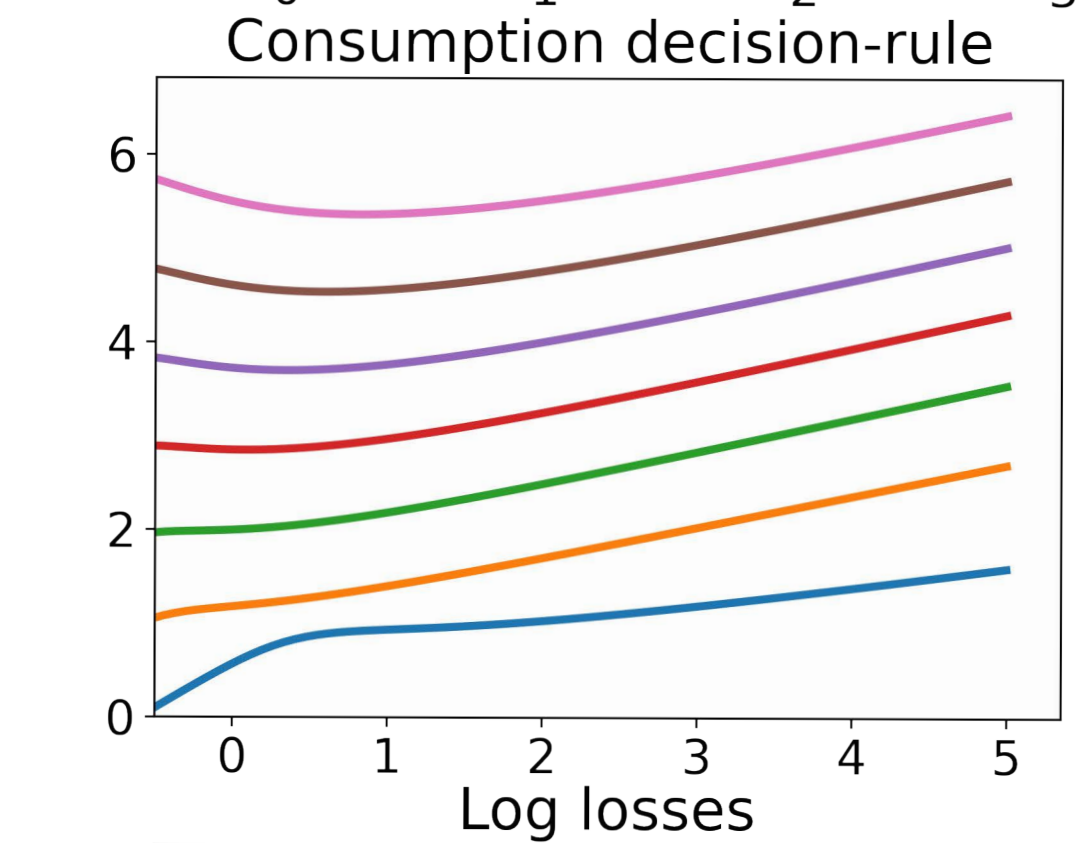
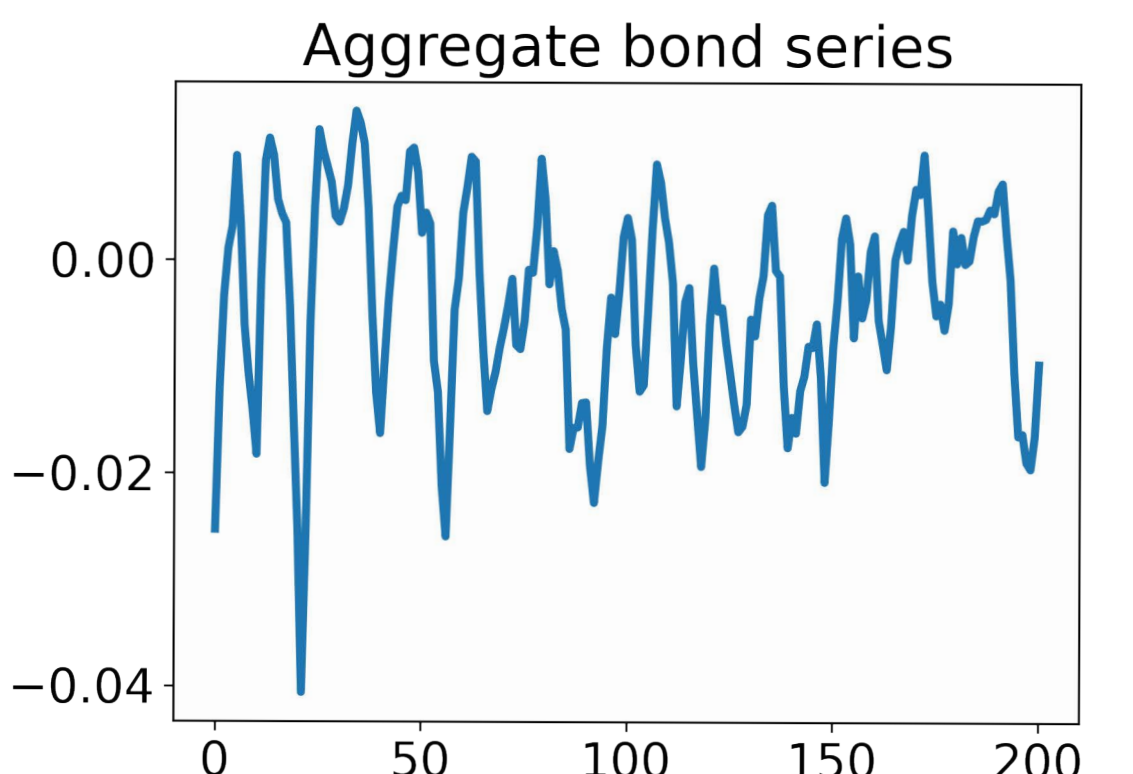
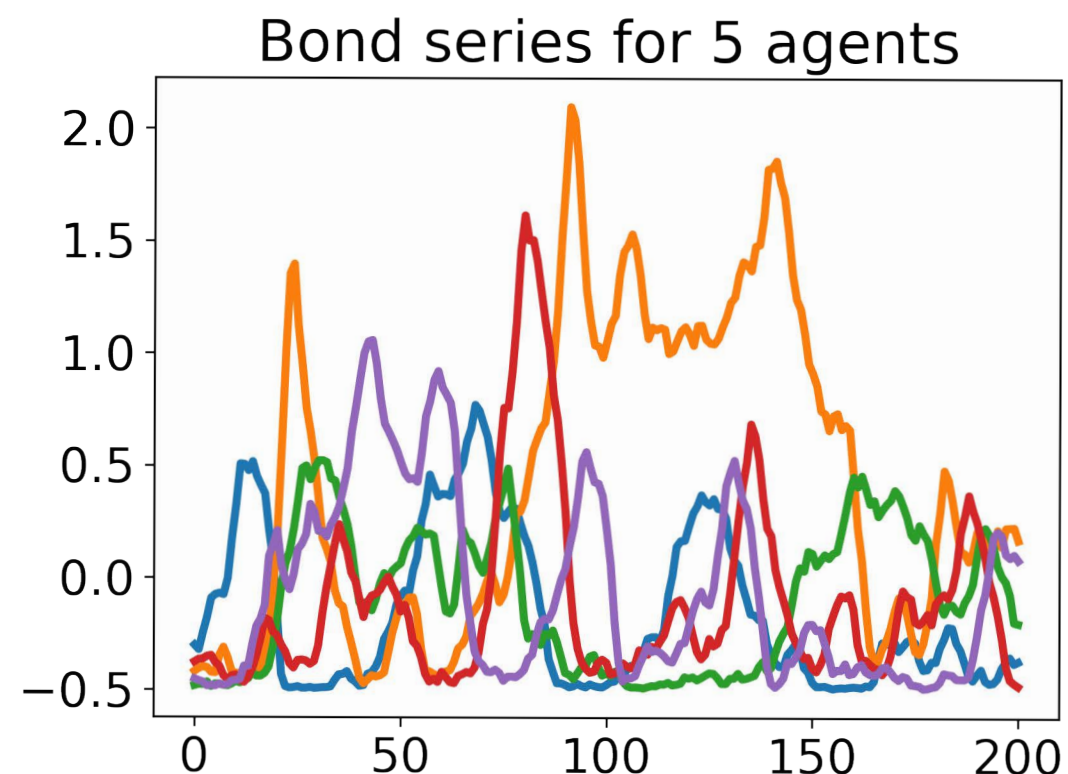
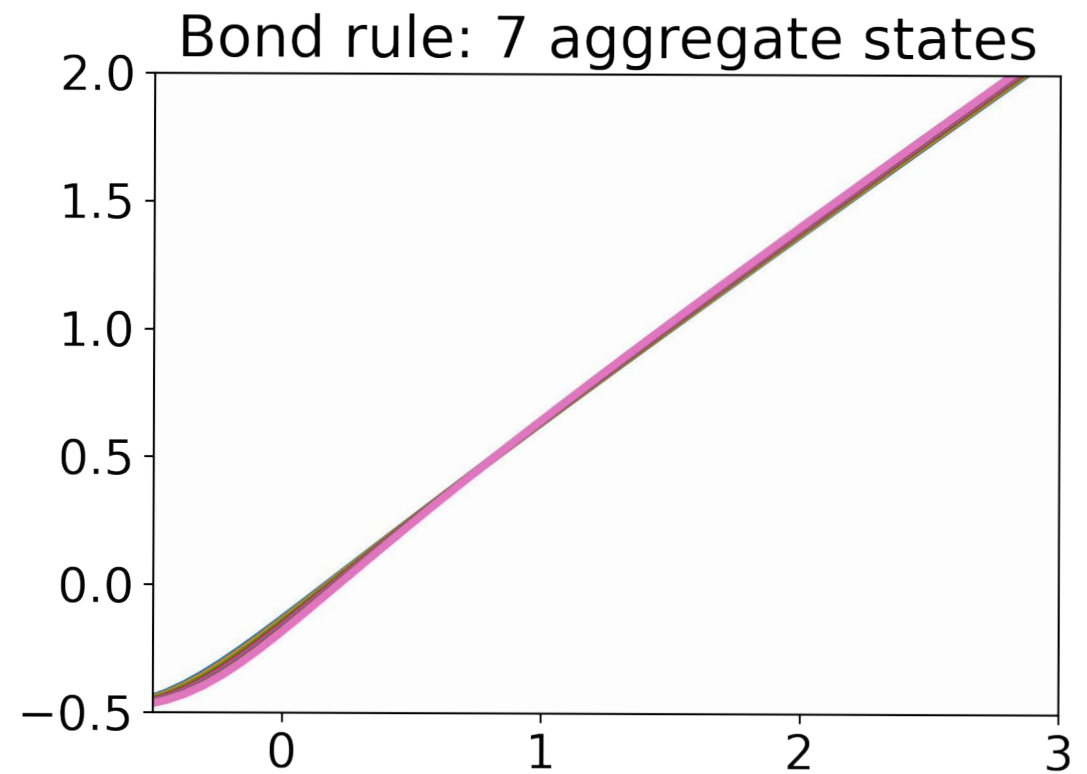
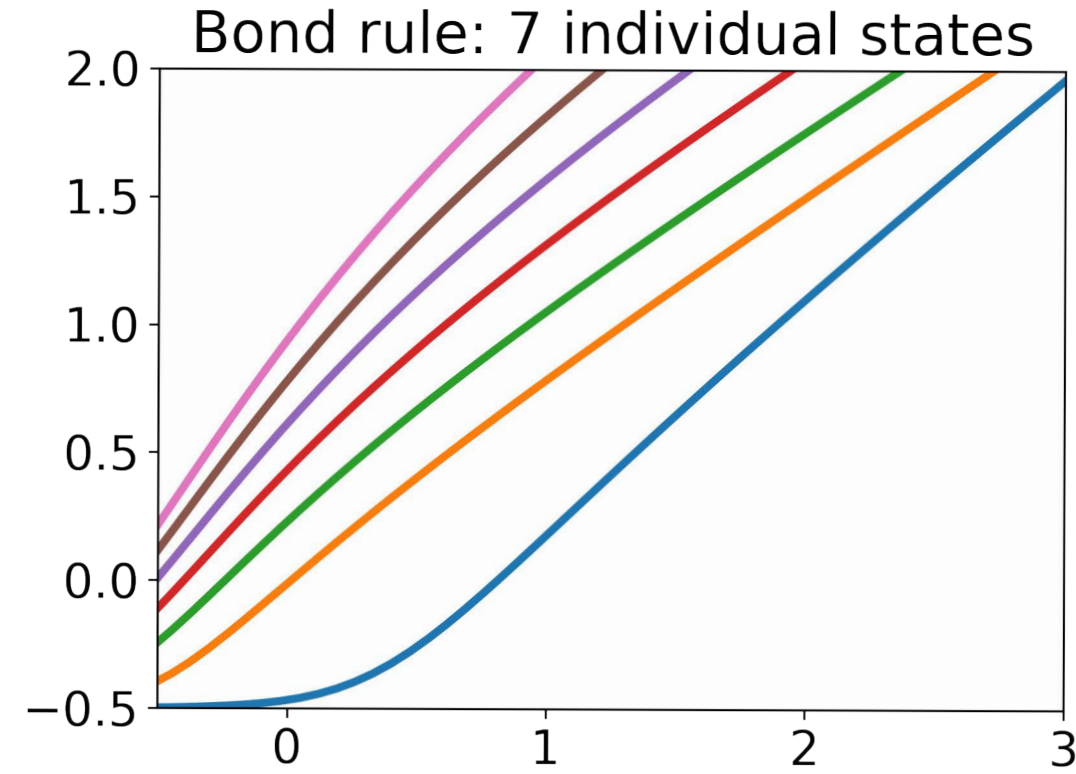
- the model with savings through bonds represents an additional complication compared to the baseline Krusell and Smith (1998) economy with capital.
- Specifically, we need to approximate the bond price function q with a different neural network

$$q_t = Q \left(\{b_t^i, y_t^i\}_{i=1}^{\ell}, z_t; \theta \right).$$

- But we did not need to approximate the prices in the HANK model with capital since they given explicitly $R_t = 1 - d + z_t \alpha k_t^{\alpha-1} h_t^{1-\alpha}$ and $W_t = z_t (1 - \alpha) k_t^{\alpha} h_t^{-\alpha}$.
- The presence of q_t brings a complication: when we simulate the model with ℓ agents, we produce ℓ data points for individual variables but just one point for the aggregate price.
- We use batches of the size N , i.e., we repeat the simulation $N =$ times to get sufficient data for approximating q .
- So, we had $N \times \ell$ observations on agents and N observations on the aggregate economy.

Calibration

- In the benchmark case, we parameterize the model by $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ with a risk-aversion coefficient of $\gamma = 1$, and assuming $\beta = 0.96$; $\rho = 0.9$; $\sigma = 0.07$; $\rho_z = 0.9$; and $\sigma_z = 0.4(1 - \rho_z^2)^{1/2}$.
- We perform training using the *ADAM* stochastic gradient descent method with the batch size of 100 and the learning rate of 0.001.
- We fix the number of iterations (which is also a simulation length) to be $K = 500,000$.
- But we perform the training only each 10th step, so the effective number of iterations is 50,000.
- The choice of these parameters must ensure both convergence and low running time and it reflects our experience in constructing deep learning approximations.
- Finally, we study numerically the role of the elasticity γ of the utility function by performing a sensitivity analysis..
- We perform training using the *ADAM* method with the batch size of 10 and the learning rate of 0.001.



Krusell and Smith (1998) method with bonds and capital

- The solutions with moments and actual state space look relatively close, in particular, the simulated series are relatively close in all 3 cases.
- Can we use Krusell and Smith (1998) analysis?
- With savings through capital, Krusell and Smith (1998) parameterized the individual and aggregate decision functions by

$$\begin{aligned}c_t^i &= C(k_t^i, y_t^i, z_t, K_t), \\ \ln K_{t+1} &= b_0 + b_1 \ln z_t + b_2 \ln K_t,\end{aligned}$$

They find that R^2 of the second regression is of order 0,999999.

- So let's try to implement the same approach in the economy with bonds by using

$$\begin{aligned}c_t^i &= C(b_t^i, y_t^i, z_t, m_t), \\ \ln m_{t+1} &= b_0 + b_1 \ln z_t + b_2 \ln m_t,\end{aligned}$$

where m_t is a selected set of moments (statistics).

Why can Krusell and Smith (1998) method be inaccurate?

- First, it could be that the selected set of moments is not sufficient to capture the information about the distributions. However, our companion figure 3 shows that the second moment solutions obtained are not too far from the solution obtained under the actual decision rules.
- Second, the aggregate law of motion is inaccurate (recall that ALM is unnecessary under our DL method but indispensable under Krusell and Smith (1998) analysis. We assess the regression with 2 moments

$$c_t^i = C(k_t^i, y_t^i, z_t, \bar{m}_1, s_1, s_2.s_{12}),$$

$$\ln \bar{m}_{1,t+1} = b_0 + b_1 \ln z_t + b_2 \ln (\bar{m}_1, s_1, s_2.s_{12}), R^2 = 0.832$$

$$\ln \bar{s}_{1,t+1} = b_0 + b_1 \ln z_t + b_2 \ln (\bar{m}_1, s_1, s_2.s_{12}), R^2 = 0.979$$

$$\ln s_{2t+1} = b_0 + b_1 \ln z_t + b_2 \ln (\bar{m}_1, s_1, s_2.s_{12}), R^2 = 0.663$$

$$\ln s_{12t+1} = b_0 + b_1 \ln z_t + b_2 \ln (\bar{m}_1, s_1, s_2.s_{12}), R^2 = 0.526$$

where $\bar{m}_{1,t+1}$ is mean of idiosyncratic shocks and $s_1, s_2.s_{12}$ are second moments of shocks and cross term.

- Thus, the explanatory power of the Krusell and Smith (1998) type of regressions is insufficient.

Heterogenous agent new Keynesian model (HANK)

- We consider a stylized HANK model with savings through bonds:

$$\begin{aligned} & \max_{\{c_t^i, n_t^i, b_{t+1}^i\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u \left(c_t^i, \ell_t^i \right) \right] \\ \text{s.t. } & c_t^i + b_t^i = \frac{R_{t-1}}{\pi_t} b_{t-1}^i + w_t y_t^i n_t^i + \Pi_t, \\ & b_{t+1}^i \geq \bar{b}, \end{aligned}$$

where c_t^i is consumption, $\ell_t^i + n_t^i = 1$, $w_t = z_t$, where R_{t-1} is nominal interest rate; π_t is inflation; and Π_t is the profit distributed equally across the agents.

- Output depends on aggregate labor H_t , aggregate productivity shock z_t and on a distortion from inflation Δ_t^p :

$$Y_t = z_t H_t \Delta_t^p$$

Heterogenous agent new Keynesian model (HANK)

- The real wage is determined by marginal cost m_t according to $w_t = z_t m_t$. In turn, marginal cost m_t is determined by the Phillips curve

$$m_t = \frac{1}{\mu} + \frac{1}{\mu\kappa} \left\{ \pi_t (\pi_t - \pi^*) - E_t \left[\Lambda_{t+1} \pi_{t+1} (\pi_{t+1} - \pi^*) \frac{Y_{t+1} \Delta_t^p}{Y_t \Delta_{t+1}^p} \right] \right\}$$

where where $c_t^i = \left[\frac{\psi(1-\ell_t^i)^{-\eta}}{w_t} \right]^{-\frac{1}{\gamma}}$ and

$$\Lambda_{t+l} \equiv \beta^l \frac{E_t \left[\int_{j \in U_{t+l}^{illiq}} (c_{t+l}^i)^{-\gamma} di \right]}{\int_{j \in U_t^{illiq}} (c_t^i)^{-\gamma} dj} \text{ with } U_t^{illiq} \text{ being the set of agents}$$

who are not against their illiquid asset borrowing constraint an μ and κ are the parameters.

Heterogenous agent new Keynesian model (HANK)

- The profit is given by the difference between output and labor income $\Pi_t \equiv Y_t - w_t H_t$.
- The distortion is determined by how much the inflation π_t differs from its target level π^* :

$$\Delta_t^p \equiv 1 - \frac{1}{2\kappa(\mu - 1)} (\pi_t - \pi^*)^2,$$

where κ and μ are the parameters.

- Finally, the nominal interest rate is determined by the Taylor rule

$$R_t \equiv R_* \left(\frac{\pi_t}{\pi_*} \right)^{\phi_\pi} \left(\frac{Y_t}{Y_*} \right)^{\phi_y}, \quad (1)$$

where R_t and R_* are the gross nominal interest rate at t and its long-run value, respectively; π_* is the target inflation; where ϕ_π and ϕ_y are the parameters in the Taylor rule. We assume that the interest rate satisfies the zero or effective lower bound ZLB or ELB respectively.

The solution procedure for HANK

- We solve the HANK model by using a similar DL solution method.
- As in the, the model has $2\ell + 1$ state variables $\{b_t^i, y_t^i\}_{i=1}^{\ell}, z_t$.
- We parameterize with neural network

aggregate variables m_t, π_t in terms of $\left(b_t^i, y_t^i, \{b_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right)$

individual variable c_t^i, ℓ_t^i in terms of $\left(\{b_t^i, y_t^i\}_{i=1}^{\ell}, z_t\right)$

The solution procedure for HANK (cont.)

We perform the following sequence of computations:

1. Given m_t , find

$$w_t = m_t z_t$$

2. Find profits

$$\Pi_t = Y_t - w_t H_t$$

Alternatively, $\Pi_t = \frac{Y_t}{\Delta_t^p} (1 - m_t) - \frac{1}{2\kappa(\mu-1)} (\pi_t - \pi^*)^2 \frac{Y_t}{\Delta_t^p}$. Also, we have $Y_t \frac{m_t}{\Delta_t^p} = w_t H_t$.

3. Given π_t , find

$$\Delta_t^p \equiv 1 - \frac{1}{2\kappa(\mu-1)} (\pi_t - \pi^*)^2$$

4. Find

$$Y_t = z_t H_t \Delta_t^p$$

5. Find the nominal interest rate from the Taylor rule

$$R_t \equiv R_* \left(\frac{\pi_t}{\pi_*} \right)^{\phi_\pi} \left(\frac{Y_t}{Y_*} \right)^{\phi_y}.$$

Loss functions for training

We next formulate the loss functions for training:

1. The Phillips curve provides the condition that identifies the marginal cost m_t

$$m_t \mu - \frac{1}{\kappa} \left\{ \pi_t (\pi_t - \pi^*) - E_t \left[\tilde{\Lambda}_{t,t+l} \pi_{t+1} (\pi_{t+1} - \pi^*) \frac{Y_{t+1} \Delta_t^p}{Y_t \Delta_{t+1}^p} \right] \right\} = 1$$

2. The equation that identifies π_t is the market clearing condition

$$\int_0^1 b_t(j) di = 0$$

3. The intratemporal first-order condition identifies labor

$$n_t^i = 1 - \left[\frac{c_i^{-\gamma} w_t y_t^i}{B} \right]^{-1/\eta}$$

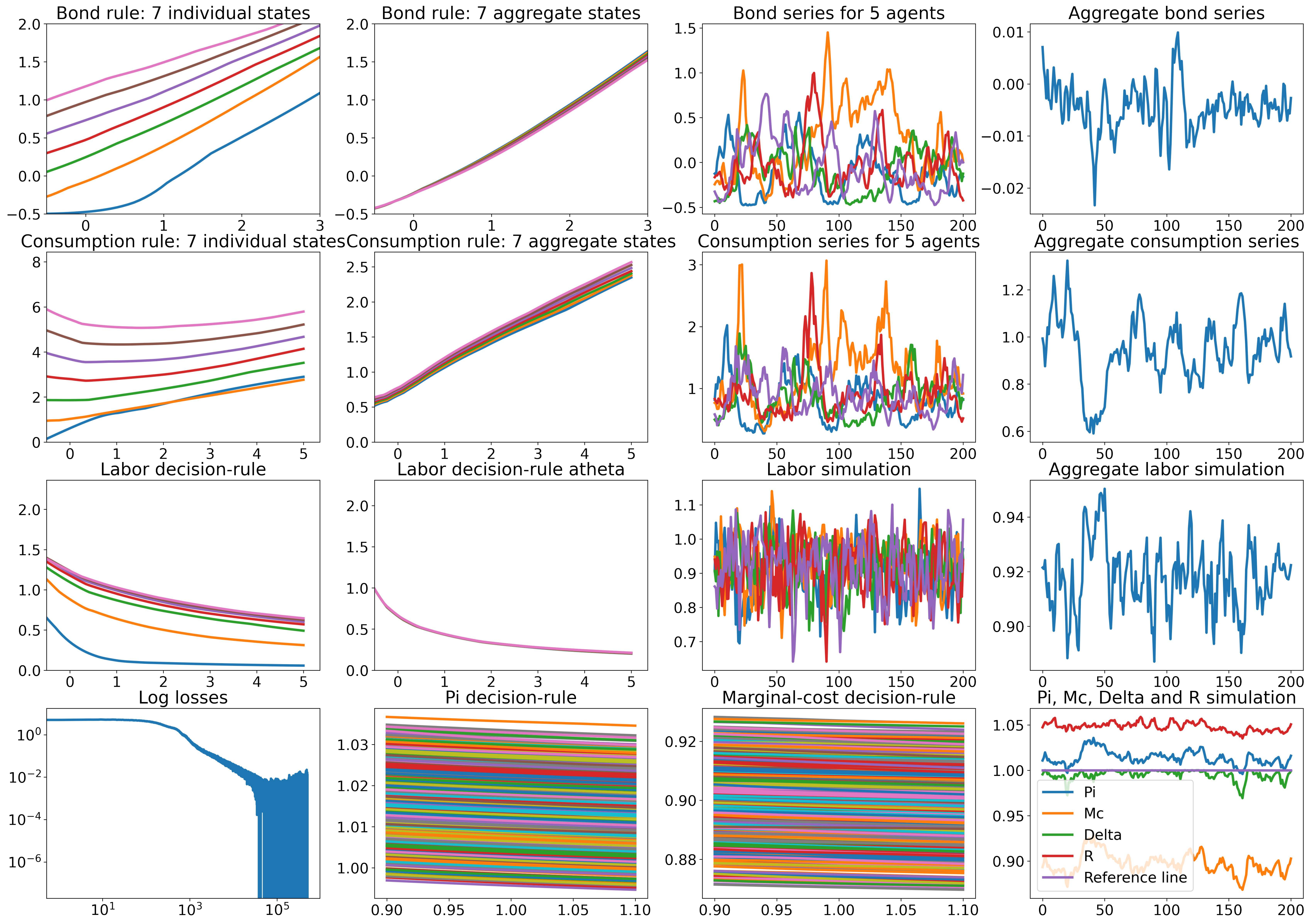
4. The intertemporal first-order condition identifies the consumption and savings choices

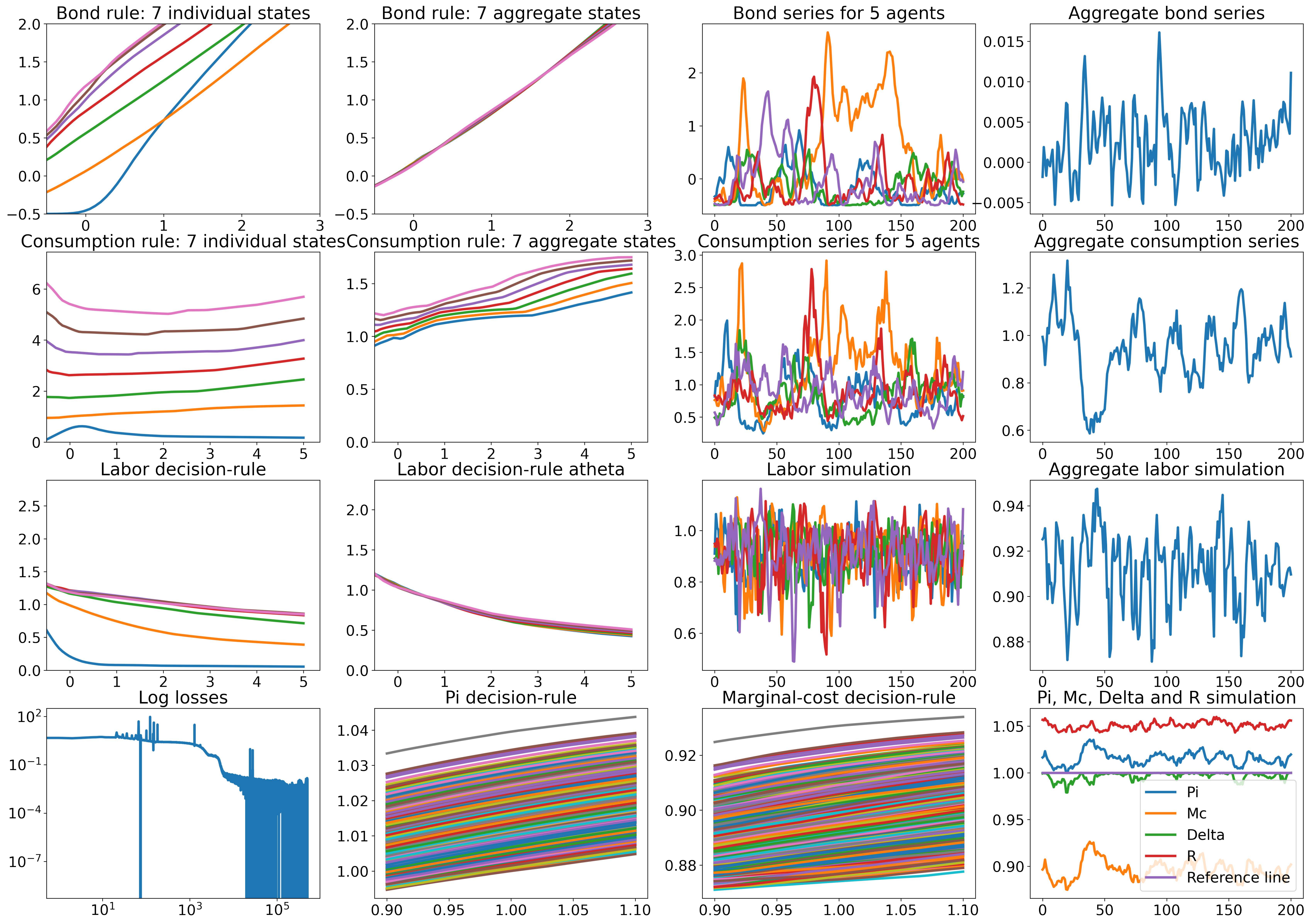
$$\Psi^{FB} \left\{ \frac{w^i}{c^i} - 1, \eta^i - 1 \right\} = 0$$

Calibration

The following values and targets are from Debortoli and Gali (2018)

Parameter	Description	Target/Source
$\gamma = 1$	Risk aversion	standard
$\eta = 1$	Labor supply elasticity	standard
$\beta = .975$	Discount factor	
$d = .1$	Depreciation rate	
$\alpha = \frac{1}{3}$	Capital share	standard
$\mu = \frac{10}{9} -$	Elasticity of substitution among goods	profits share of 10%
$\kappa = \frac{9}{105}$	Price adjustment cost	avg. price duration
$\bar{b} = -0.4$	Liquid asset borrowing constraint	75% of people have
$\rho_z = 0.9777$	Persistence of idiosyncratic shock	persistence of annual
$\sigma_z = 0.1928$	Standard deviation of idiosyncratic shock	standard deviation of
$\rho_y = 0.9$	Persistence of TFP shock	standard
$\sigma_y = 0.016$	Standard deviation of TFP shock	standard

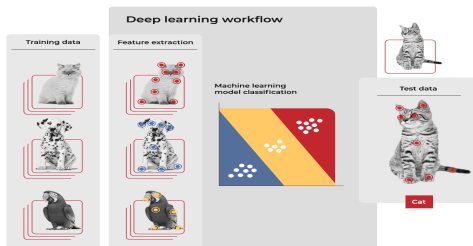




Part 2: Modeling discrete-set choices

An illustration from data science: image recognition

- Consider the image recognition problem—a typical classification problem in data science.
- For example, a machine classifies images into cats, dogs and birds.
- We parameterize the probabilities of the three classes with a deep neural network.



- The machine is given a collection of images and is trained to minimize the cross-entropy loss (which is equivalent to maximizing the likelihood function) that ensures the correct classification of images; see Goodfellow, Bengio and Courville (2016) for a survey of classification methods in data science.

Classification method for discrete choices in economics

- Our classification method in macroeconomics is analogous to the above image-recognition analysis.
- For example, we use a deep neural network to parameterize the probabilities of being full-time employed, part-time employed and unemployed.



- The machine is given a collection of employment choices conditional on state and is trained to maximize the likelihood function that those choices are optimal.
- The same idea can be applied for analyzing the models with retirement, default, house purchase, etc.

Krusell and Smith's (1999) model

- Deep learning can be used to solve small-scale representative agent models.
- However, the power of deep learning consists in its ability to solve large-scale applications intractable with conventional solution methods.
- To illustrate this power, we solve Krusell and Smith's (1998) model in which the agents face indivisible labor choices, specifically:
 - *a version model with continuous choices (i.e., divisible labor);*
 - *an indivisible-labor version with 2 discrete labor states (employed and unemployed);*
 - *an indivisible-labor version with 3 discrete labor states (employed, unemployed and part-time employed agent)*

The model

- Heterogeneous agents $i = 1, \dots, \ell$. Each agent i solves

$$\begin{aligned} \max_{\{c_t^i, k_{t+1}^i, n_t^i\}_{t=0}^{\infty}} \quad & E_0 \left[\sum_{t=0}^{\infty} \beta^t u \left(c_t^i, n_t^i \right) \right] \\ \text{s.t.} \quad & c_t^i + k_{t+1}^i = R_t k_t^i + W_t v_t^i n_t^i, \\ & n_t \in N, \\ & \ln v_{t+1}^i = \rho_v \ln v_t^i + \sigma_v \epsilon_t^i \text{ with } \epsilon_t^i \sim \mathcal{N}(0, 1), \\ & k_{t+1}^i \geq \bar{k}, \end{aligned}$$

where c_t^i , n_t^i , k_t^i and v_t^i are consumption, hours worked, capital and idiosyncratic labor productivity; $\beta \in (0, 1)$ is the discount factor; $\rho_v \in (-1, 1)$ and $\sigma_v \geq 0$; and initial condition (k_0^i, v_0^i) is given. The capital choice is restricted by a borrowing limit $\bar{k} \leq 0$.

Production side

- The production side of the economy is described by a Cobb-Douglas production function $\exp(z_t) k_t^{\alpha-1} h_t^{1-\alpha}$, where $k_t = \sum_{i=1}^{\ell} k_t^i$ is aggregate capital, $h_t = \sum_{i=1}^{\ell} v_t^i n_t^i$ is aggregate efficiency labor, and z_t is an aggregate productivity shock following a first-order autoregressive process,

$$\ln z_{t+1} = \rho_z \ln z_t + \sigma_z \epsilon_t \text{ with } \epsilon_t \sim \mathcal{N}(0, 1),$$

where $\rho_z \in (-1, 1)$ and $\sigma_z \geq 0$.

- The interest rate R_t and wage W_t are given by

$$R_t = 1 - d + z_t \alpha k_t^{\alpha-1} h_t^{1-\alpha} \text{ and } W_t = z_t (1 - \alpha) k_t^{\alpha} h_t^{-\alpha},$$

where $d \in (0, 1]$ is the depreciation rate.

Three different version of the model

We consider three versions of the model that differ in the set of allowable labor choices $n_t \in N$:

- i) divisible labor model $N = [0, L],$
- ii) indivisible labor model $N = \{0, \bar{n}\},$
- iii) three-state employment model $N = \{0, \underline{n}, \bar{n}\},$

Deep learning method for divisible labor model

Divisible labor model

- To characterize labor choice, we assume that the utility function takes the form

$$u(c, n) = \frac{c^{1-\gamma} - 1}{1-\gamma} + B \frac{(L-n)^{1-\eta} - 1}{1-\eta},$$

where $\gamma, \eta > 0$ and L is the total time endowment.

- We normalize time to L instead of the conventional normalization to 1 because it helps to calibrate the divisible and indivisible labor models to the same steady state.
- The labor choice is characterized by a FOC

$$n_t^i = L - \left[\frac{c_i^{-\gamma} W_t v_t^i}{B} \right]^{-1/\eta}$$

where v_t^i is an idiosyncratic labor productivity shock.

Training errors and running time

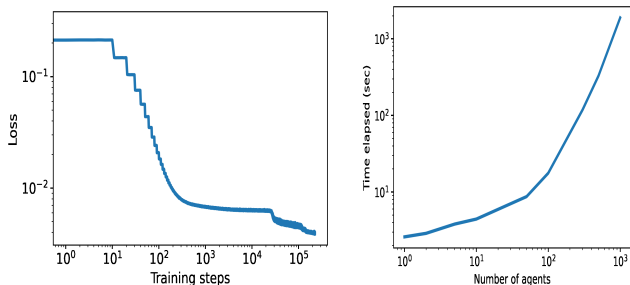


Figure 1. Training errors and running time for divisible labor model.

The solution for divisible labor model

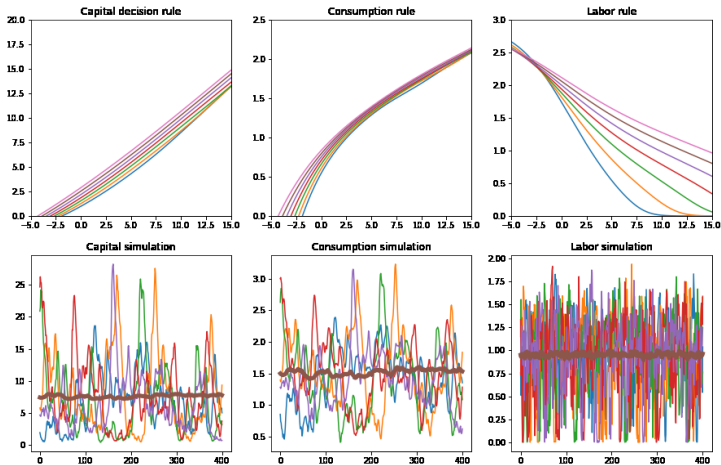


Figure 2. Solution to divisible labor mode.

Deep learning method for indivisible labor model

Indivisible labor model with 2 states

The agent chooses to be employed ($n_t^i = \bar{n}$) or unemployed ($n_t^i = 0$) depending on which of the two choices leads to a higher continuation value, i.e.,

$$\begin{aligned} n_t^i &= \bar{n} \text{ if } V^E = \max \{V^E, V^U\} \\ n_t^i &= 0 \text{ otherwise.} \end{aligned}$$

where V^E and V^U denote value functions of the agent in the employed and unemployed states, respectively.

Logistic regression

Let us consider a typical classification problem. We have ℓ data points $\{X^i, y^i\}_{i=1}^{\ell}$ where $X^i \equiv (1, x_1^i, x_2^i, \dots)$ is a collection of dependent variables (features) and y^i is a categorical independent variable (label) that takes values 0 and 1. The goal is to construct a dashed line that separates the known examples of the two types.

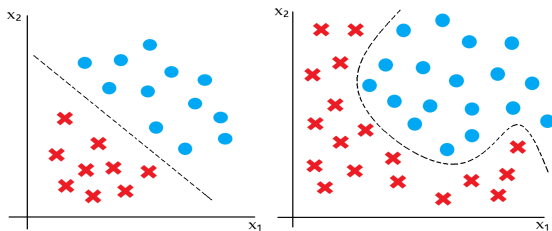


Figure 3. Examples of binary classification.

We restrict attention to one technique – logistic regression – which is simple, general and can be conveniently combined with our deep learning analysis.

A hypothesis

As a first step, we form a hypothesis about the functional form of the separating line. For the left panel, it is sufficient to assume that the separating line is linear

$$H_0 : \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0,$$

but for the right panel, we must use a sufficiently flexible nonlinear separating function such as a higher-order polynomial function,

$$H_0 : \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1 x_2 + \theta_5 x_2^2 + \dots = 0,$$

where $(\theta_0, \theta_1, \dots) \equiv \theta$ are the polynomial coefficients. When $X\theta \equiv \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots > 0$, we conclude that y belongs to class 1 and otherwise, we conclude that it is from class 0.

Estimation

- Our next step is to estimate θ coefficients. Since y is a categorical variable $y \in \{0, 1\}$, we cannot use ordinary least-squares estimator, i.e., we cannot regress y on $X\theta$. Instead, we form a logistic regression

$$H_0 : \log \frac{p}{1-p} = X\theta,$$

where p is the probability that a data point with characteristics $X \equiv (1, x_1, x_2, x_1^2, \dots)$ belongs to class 1, and $\theta \equiv (\theta_0, \theta_1, \dots, \theta_m, \dots, \theta_M)$ is a coefficient vector.

- The logistic function is an excellent choice for approximating probability:
 - First, it ensures that $p = \frac{1}{1+\exp(-X\theta)} \in (0, 1)$ for any θ and X , and hence p and $(1-p)$ can be interpreted as probabilities that a data point belongs to classes 1 and 0, respectively.
 - Second, $p = \frac{1}{2}$ corresponds to the separation line $X\theta = 0$. Hence, when $p > \frac{1}{2}$, the data point is "above" the separating line $X\theta$, and thus, belongs to the class 1 and if $p < \frac{1}{2}$, the opposite is true.
 - Finally, when $X\theta \rightarrow -\infty$ and $X\theta \rightarrow +\infty$, we have that $p \rightarrow 0$ and $p \rightarrow 1$, respectively.

Probability of an observation

The logistic regression provides a convenient way to estimate the decision boundary coefficients θ by using a maximum likelihood estimator. A probability that the data point i belongs to classes 0 and 1 can be represented with a single formula by

$$\text{Prob}(y \mid X; \theta) = p^y (1 - p)^{1-y}.$$

Indeed, if $y = 1$, we have $\text{Prob}(y = 1 \mid X; \theta) = (p)^1 (1 - p)^0 = p$; and if $y = 0$, we have $\text{Prob}(y = 0 \mid X; \theta) = (p)^0 (1 - p)^1 = 1 - p$.

Likelihood function

We search for the coefficient vector θ that maximizes the (log)likelihood of the event such that a given matrix of features $\{X^i\}_{i=1}^{\ell}$ produces the given output realizations $\{y^i\}_{i=1}^{\ell}$, i.e.,

$$\begin{aligned}\max_{\theta} \ln L(\theta) &= \ln \prod_{i=1}^{\ell} (p(X^i; \theta))^{y^i} (1 - p(X^i; \theta))^{1-y^i} = \\ &\sum_{i=1}^{\ell} [y^i \ln(p(X^i; \theta)) + (1 - y^i) \ln(1 - p(X^i; \theta))],\end{aligned}$$

where the probability $p(X^i; \theta) \equiv \frac{1}{1 + \exp(-X^i \theta)}$ is given by a logistic function.

Constructing a maximizer

To find the maximizer, we compute the first-order conditions with respect to all coefficients θ_m for $m = 0, \dots, M$,

$$\begin{aligned}\frac{\partial \ln L(\theta)}{\partial \theta_m} &= \sum_{i=1}^{\ell} \left[\frac{y^i}{p(X^i; \theta)} \frac{\partial p(X^i; \theta)}{\partial \theta_m} - \frac{(1 - y^i)}{(1 - p(X^i; \theta))} \frac{\partial p(X^i; \theta)}{\partial \theta_m} \right] \\ &= \sum_{i=1}^{\ell} [y^i x_m^i (1 - p(X^i; \theta)) - (1 - y^i) x_m^i p(X^i; \theta)] \\ &= \sum_{i=1}^{\ell} [y^i - p(X^i; \theta)] x_m^i,\end{aligned}$$

where x_m^i is the feature m of agent i .

The constructed gradient $\nabla \ln L_{\theta}(\theta) \equiv \left[\frac{\partial \ln L(\theta)}{\partial \theta_1}, \dots, \frac{\partial \ln L(\theta)}{\partial \theta_M} \right]'$ can be used for implementing the gradient descent-style method

$$\theta \leftarrow \theta - \lambda \nabla \ln L_{\theta}(\theta).$$

Decisions in divisible versus indivisible labor

- In the divisible labor model, we construct a policy function that determines the hours worked $\frac{n_t^i}{L}$.
- In the indivisible labor model studied here, we construct a decision boundary $\varphi(s_t^i; \theta) = 0$ that separates the employment and unemployment choices conditional on state $s_t^i \equiv \left(k_t^i, v_t^i, \{k_t^i, v_t^i\}_{i=1}^\ell, z_t\right)$.
- Whenever $\varphi(s_t^i; \theta) \geq 0$, the agent is employed $n_t^i = \bar{n}$ and otherwise, the agent is unemployed $n_t^i = 0$.
- Let us show how such a decision boundary can be constructed by using the logistic regression classification method.

Decisions in divisible versus indivisible labor

- Since our model has a large number of explanatory variables (state variables) as well as a highly nonlinear decision boundary, we use neural networks for approximating such boundary (instead of the polynomial function).
- We estimate the coefficients of the neural network (weights and biases) by formulating a logistic regression,

$$H_0 : \log \frac{p}{1-p} = \varphi(s; \theta).$$

- We parameterize the decision function for p_t^i and by a sigmoid function in the indivisible labor model:

$$\sigma \left(\zeta_0 + \varphi \left(k_t^i, v_t^i, \{k_t^i, v_t^i\}_{i=1}^{\ell}, z_t; \theta \right) \right),$$

where $\varphi(\cdot)$ is a multilayer neural network parameterized by a vector of coefficients θ (weights and biases), $\sigma(z) = \frac{1}{1+e^{-z}}$ is a sigmoid function which ensures that p_t^i is bounded in the interval $[0, 1]$, respectively, and ζ_0 is a constant term.

Decisions in divisible versus indivisible labor

- The function p_t^i , allows us to infer the indivisible labor choice directly, specifically, an agent is employed $n_t^i = \bar{n}$ whenever $p_t^i \geq \frac{1}{2}$ and is unemployed otherwise $n_t^i = 0$.
- We can then compute $h_t = \sum_{i=1}^{\ell} v_t^i n_t^i$ and find W_t and R_t restore the remaining individual and aggregate variables.
- Our next goal is to check if the constructed labor choices are consistent with the individual optimality conditions.
- We use the decision function p_t^i to restore the value functions for the employed and unemployed agents $V^E(s_t^i; \theta^E)$ and $V^U(s_t^i; \theta^U)$.
- We next construct the labor choice $\hat{n}_t^i$ implied by these two value functions

$$\hat{n}_t^i = \begin{cases} \bar{n} & \text{if } V^E = \max \{V^E, V^U\}, \\ 0 & \text{otherwise.} \end{cases}$$

Decisions in divisible versus indivisible labor

In the solution, the labor choice $\widehat{n}_t^i$ implied by the value functions must coincide with the labor choice n_t^i produced by our decision function for all i and t . If this is not the case, we proceed with training our classifier. To this purpose, we construct the categorical variable $y_t^i \in \{0, 1\}$ such that

$$y_t^i = \begin{cases} 1 & \text{if } \widehat{n}_t^i = \bar{n}, \\ 0 & \text{otherwise,} \end{cases}$$

and we use it to form the (log)likelihood function

$$\ln L(\theta) = \frac{1}{\ell} \sum_{i=1}^{\ell} [y_t^i \ln(p(s_t^i; \theta)) + (1 - y_t^i) \ln(1 - p(s_t^i; \theta))].$$

We then maximize the likelihood function by using a conventional / stochastic / batch stochastic gradient descent methods. We iterate on the decision function p_t^i until convergence.

Training errors and running time

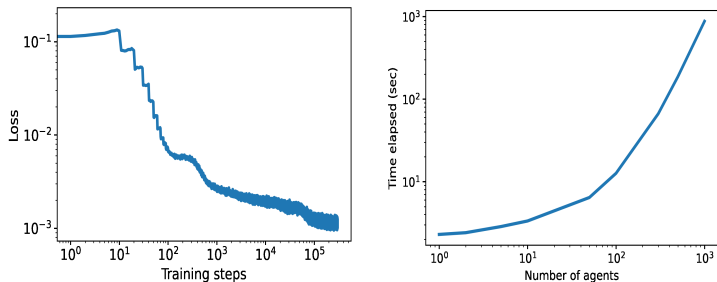


Figure 7. Training errors and running time for indivisible labor model.

The solution for divisible labor model

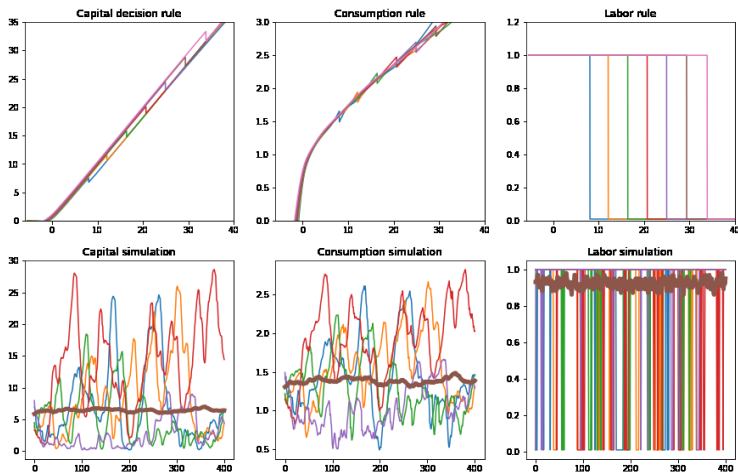


Figure 8. Solution to indivisible labor model under $\gamma = 1$ and $\eta = 1$.

Deep learning method for the model with 3 states

Indivisible labor model with 3 states

The three employment states, $n_t^i = \bar{n}$, $n_t^i = \underline{n}$ and $n_t^i = 0$, correspond to full-time unemployment, part-time employment and unemployment, respectively,

$$\begin{aligned}n_t^i &= \bar{n} \text{ if } V^{FT} = \max \{V^U, V^{FT}, V^{PT}\} \\n_t^i &= \underline{n} \text{ if } V^{PT} = \max \{V^U, V^{FT}, V^{PT}\} \\n_t^i &= 0 \text{ otherwise}\end{aligned}$$

where V^{FT} , V^{PT} and V^U denote value functions of full-time employed, part-time employed and unemployed agents, respectively.

Multiclass classification problem

We again have a collection of ℓ data points $\{X^i, y^i\}_{i=1}^{\ell}$ where $X^i \equiv (1, x_1^i, x_2^i, \dots)$ is composed of dependent variables (features) but now y^i is a categorical independent variable (label) that takes K values. Our goal is to construct the lines that separate the classes 1, 2 and 3.

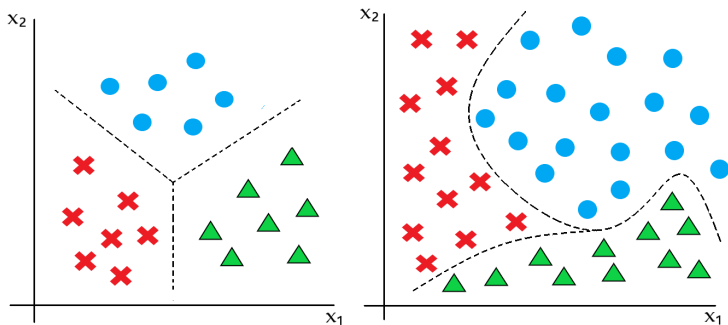


Figure 9. Examples of multiclass classification.

From multiclass to binary classification problem

- A popular approach in machine learning is to reformulate a multiclass classification problem as a collection of binary classification problems.
- The key assumption behind this approach is the hypothesis of an independence of irrelevant alternatives.
- In our analysis, that means that the choice between $\{\times\}$ and $\{\triangle\}$ is independent of the availability of $\{\circ\}$, the choice between $\{\triangle\}$ and $\{\circ\}$ is independent of the availability of $\{\times\}$ and the choice between $\{\circ\}$ and $\{\times\}$ is independent of the availability of $\{\triangle\}$.
- Two binary reformulations of a multiclass classification problems are the *one-versus-one* and *one-versus-rest* (or *one-versus-all*) classifiers,

$$\ln \frac{p(\times)}{p(\circ)} = X\theta^{(1)} \quad \ln \frac{p(\triangle)}{p(\circ)} = X\theta^{(2)} \quad \ln \frac{p(\triangle)}{p(\times)} = X\theta^{(3)},$$

$$\ln \frac{p(\times)}{p(\circ)+p(\triangle)} = X\theta^{(1)} \quad \ln \frac{p(\triangle)}{p(\circ)+p(\times)} = X\theta^{(2)} \quad \ln \frac{p(\circ)}{p(\triangle)+p(\times)} = X\theta^{(3)},$$

where $\theta^{(1)}$, $\theta^{(2)}$ and $\theta^{(3)}$ are the regression coefficients and X is the matrix of features.

Training multi class classifiers

- To train the constructed multiclass classifiers, we may omit one of three regressions by imposing the restriction that the probabilities are added to one.
- For the one-versus-one classifier, the first two regressions imply $p(\times) = p(\bullet) \exp(X\theta^{(1)})$ and $p(\Delta) = p(\bullet) \exp(X\theta^{(2)})$ so that $p(\bullet) \left(1 + \exp(X\theta^{(1)}) + \exp(X\theta^{(2)})\right) = 1$.
- In turn, for the one-versus-rest classifier, in the first regression, we replace $p(\bullet) + p(\Delta)$ with $1 - p(\times)$ and in the second regression, we replace $p(\bullet) + p(\times)$ with $1 - p(\Delta)$.

Consequently, we can re-write two classifiers as

$$p(\times) = \exp(X\theta^{(1)}) p(\bullet), \quad p(\Delta) = \exp(X\theta^{(2)}) p(\bullet), \\ p(\bullet) = \frac{1}{1 + \exp(X\theta^{(1)}) + \exp(X\theta^{(2)})},$$

$$p(\times) = \frac{1}{1 + \exp(-X\theta^{(1)})} \quad p(\Delta) = \frac{1}{1 + \exp(-X\theta^{(2)})} \quad p(\bullet) = 1 - p(\times) - p(\Delta).$$

Symmetric one-versus-rest classifier

- Note that in the above expressions, we treat the normalizing class $\{\mathbf{o}\}$ differently from the other two classes $\{\Delta, \times\}$.
- There is also a symmetric version of the *one-versus-rest* method in which all K classes are treated identically by estimating K unnormalized one-versus-rest logistic regressions $\ln p(\times) = X\theta^{(1)}$, $\ln p(\Delta) = X\theta^{(2)}$, $\ln p(\mathbf{o}) = X\theta^{(3)}$ and by normalizing the exponential function ex-post by their sum.
- This classifier is called softmax and it is a generalization of a logistic function to multiple dimensions,

$$\begin{aligned}p(\times) &= \frac{1}{\Sigma} \exp\left(X\theta^{(1)}\right) \\p(\Delta) &= \frac{1}{\Sigma} \exp\left(X\theta^{(2)}\right) \quad , \\p(\mathbf{o}) &= \frac{1}{\Sigma} \exp\left(X\theta^{(3)}\right) ,\end{aligned}$$

where $\Sigma = \exp\left(X\theta^{(1)}\right) + \exp\left(X\theta^{(2)}\right) + \exp\left(X\theta^{(3)}\right)$.

- The symmetric treatment is convenient in deep learning analysis because it allows us to use a neural network with K symmetric outputs.

Likelihood function for softmax classifier

The log-likelihood function for the softmax classifier is similar to the one for the binary classifier except that we also do a summation over K of possible outcomes,

$$\begin{aligned} & \max_{\theta_1, \dots, \theta_K} \ln L(\theta_1, \dots, \theta_K) \\ &= \frac{1}{K\ell} \sum_{k=1}^K \sum_{i=1}^{\ell} \left[y^{i,k} \ln \left(p \left(X^i; \theta^k \right) \right) + (1 - y^{i,k}) \ln \left(1 - p \left(X^i; \theta^k \right) \right) \right], \end{aligned}$$

where $y^{i,k}$ is a categorical variable constructed so that $y^{i,k} = 1$ if observation i belongs to class k and it is zero otherwise. Again, we maximize the constructed likelihood function by using a gradient descent style method, $\theta \leftarrow \theta - \lambda \nabla \ln L_{\theta}(\theta)$.

Discrete choice in three state model

- We next extend our indivisible labor heterogeneous-agent model with two employment choices $\{0, \bar{n}\}$ to three employment choices $\{0, \underline{n}, \bar{n}\}$.
- We parameterize not one but three decision boundaries that separate the three employment choices, so we use a sigmoid function to parameterize the functions $\frac{p_t^i(\bar{n})}{\Sigma}$, $\frac{p_t^i(\underline{n})}{\Sigma}$, $\frac{p_t^i(0)}{\Sigma}$ specifically:

$$\sigma \left(\zeta_0 + \varphi \left(k_t^i, v_t^i, \{k_t^i, v_t^i\}_{i=1}^\ell, z_t; \theta \right) \right),$$

where $\varphi(\cdot)$ is a multilayer neural network parameterized by a vector of coefficients θ (weights and biases), $\Sigma \equiv p_t^i(\bar{n}) + p_t^i(\underline{n}) + p_t^i(0)$ normalizes the probabilities to one; $\sigma(z) = \frac{1}{1+e^{-z}}$ is a sigmoid function which ensures that $\frac{p_t^i(\bar{n})}{\Sigma}$, $\frac{p_t^i(\underline{n})}{\Sigma}$ and $\frac{p_t^i(0)}{\Sigma}$ are bounded in the interval $[0, 1]$, and ζ_0 is a constant term.

Verifying the optimality conditions

- Our next goal is to check if the constructed labor choices are consistent with the individual optimality conditions.
- To validate the individual choices, we use the decision functions $\frac{p_t^i(\bar{n})}{\Sigma}$, $\frac{p_t^i(\underline{n})}{\Sigma}$, $\frac{p_t^i(0)}{\Sigma}$ to recover the value functions for employed, part-time employed and unemployed agents, V^E , V^{PT} and V^U , respectively, using the appropriately formulated Bellman equations.
- We then construct the labor choice $\hat{n}_t^i$ implied by such value functions,

$$\hat{n}_t^i = \begin{cases} \bar{n} & \text{if } V^E = \max \{V^E, V^{PT}, V^U\}, \\ \underline{n} & \text{if } V^{PT} = \max \{V^E, V^{PT}, V^U\}, \\ 0 & \text{otherwise.} \end{cases}$$

- In the solution, the labor choice implied by the value function $\hat{n}_t^i$ must coincide with the labor choice produced by our decision function n_t^i for all i, t .
- If this is not the case, we proceed to training of our classifier.

Training the model

- To this purpose, we construct the categorical variable $y_t^i \equiv (y_t^{i,1}, y_t^{i,2}, y_t^{i,3})$ such that

$$y_t^i = \begin{cases} (1, 0, 0) & \text{if } \hat{n}_t^i = \bar{n}, \\ (0, 1, 0) & \text{if } \hat{n}_t^i = \underline{n}, \\ (0, 0, 1) & \text{otherwise.} \end{cases}$$

- We then formulate the (log)likelihood function

$$\begin{aligned} \ln L(\theta^{(1)}, \theta^{(2)}, \theta^{(3)}) \\ = \frac{1}{3\ell} \sum_{k=1}^3 \sum_{i=1}^{\ell} \left[\hat{y}_t^{i,k} \ln \left(p(s_t^i; \theta^{(k)}) \right) + \left(1 - \hat{y}_t^{i,k} \right) \ln \left(1 - p(s_t^i; \theta^{(k)}) \right) \right]. \end{aligned}$$

- We train the model to maximize the likelihood function by using a conventional / stochastic / batch stochastic gradient descent method.
- We iterate on the decision functions $p_t^i(\bar{n})$, $p_t^i(\underline{n})$ and $p_t^i(0)$ until convergence.

Training errors and running time

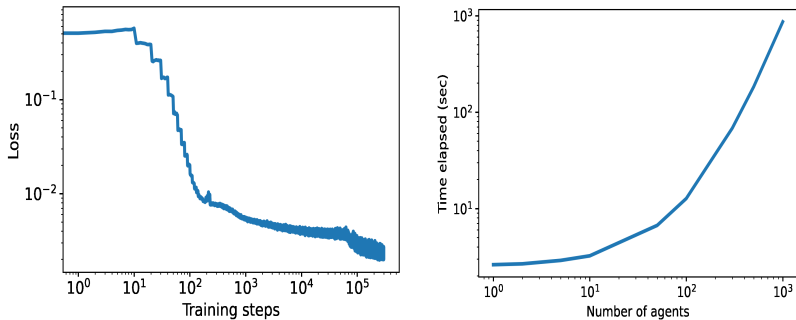


Figure 10. Training errors and running time for three-state employment m

The solution for divisible labor model

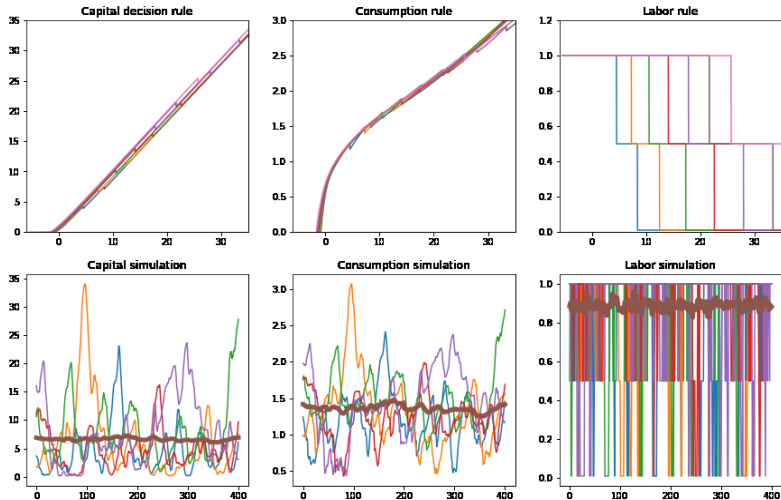


Figure 11. Solution to the three-state employment model.

Conclusion

- This paper shows how to use deep learning approach borrowed from data science for solving heterogenous-agent models.
- A combination of the state-of-the-art machine learning techniques makes the proposed method tractable in problems with very high dimensionality – hundreds and even thousands of heterogeneous agents.
- For discrete choice, we investigate just one example – discrete labor choice – but the proposed deep learning classification method has a variety of potential applications such as sovereign default models, models with retirement, and models with indivisible commodities, in particular, housing.